

claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf\_7049e1ed-b93\agent-aa95eb56290d260be.jsonl`
- SHA-256: `61fc41830efe77fc7869a8cb4123a1c6c6c182e3564f9906fd341182936353b6`
- Source modified: `2026-07-02T04:57:37+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf\_7049e1ed-b93`
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`

Transcript

1. user (2026-07-02T04:45:39.419Z)

You are a senior engineer producing ONE isolated PR. Today is 2026-07-02.

REPO: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (a shared checkout ? follow the safety rules exactly).

AUDIT FINDINGS YOU ARE FIXING: R1-18 (CLI --output-dir default clobbers config) ? Grep C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/AUDIT_REPORT_khelsutra-guru_2026-07-02.md for these IDs and read the full evidence + verifier notes before coding. TASK BRIEF:

In C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer: fix backend/main.py ~:1992 ? argparse --output-dir default currently clobbers config output.output_dir on every start. Change the argparse default to None and only override the config value when args.output_dir is not None (falling back through the existing resolve_output_dir/output_base(config) path; the documented knob is backend/config/models.py:274-276). Byte-identical default behavior when neither is set ('output'). Tests: config-only set ? honored; CLI set ? wins; neither ? 'output'. Keep the diff minimal ? this file is contested (sibling PRs touch other regions of main.py).

BRANCHING SAFETY (mandatory ? shared checkout, sibling sessions exist):

1. In the MAIN repo checkout run: git fetch origin
 2. Create an ISOLATED WORKTREE: git worktree add "C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-<shortname>" -b <branch-name> origin/master (branch explicitly FROM origin/master ? never from the current local branch; use origin/main for repos whose default is main)
 3. Do ALL work inside the worktree. Before committing: git branch --show-current + git log --oneline -1 must show YOUR branch on the origin-master base.
 4. Stage EXPLICIT paths only (git add <file> <file>) ? NEVER git add -A / . / -u
 5. Before opening the PR: git log --oneline origin/master..HEAD must list ONLY your commits.
- COMMIT/PR CONVENTIONS:

- Commit message: conventional style matching the repo's history; end the message with a blank line then: Co-Authored-By: Claude Fable 5 <noreply@anthropic.com>
- PR body: what/why, a "## Verification" section stating exactly what you ran locally and what CI covers, and end with: ? Generated with [Claude Code](https://claude.com/claude-code)
- Do NOT merge the PR. Do NOT remove the worktree (a fixer may need it). Return the PR number, branch, worktree path.

QUALITY BAR:

- Read the repo's CLAUDE.md first; for the engine also docs/CODE_MAP.md \$0 (file placement) before adding any new file.
- Add/extend tests for every behavior change. Run the targeted tests you touched (python -m pytest tests/<relevant> -q) + ruff check on changed files; if mypy config exists, run mypy on touched modules. Full suite is CI's job ? do not run it.
- Default-safe: new knobs default OFF / behavior-preserving unless the task explicitly says otherwise.
- The full audit evidence is in C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-

Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/AUDIT_REPORT_khelsutra-guru_2026-07-02.md ? Grep it for your finding IDs and READ the verifier notes; they contain corrections (exact file:line, scoping traps) you MUST honor.

If the brief's premise turns out false against current code (something already fixed, file moved), adapt honestly and say so in notes ? never force a pointless diff. If you genuinely cannot complete (env broken, auth missing), open NO PR and explain in notes with pr_number=0.

2. user (2026-07-02T04:45:47.753Z)

175-[Omitted long context line]
176-[Omitted long context line]
177-
178-### R1-15 [P2/M] test-gap ? badminton-highlight-indexer
179-**M4 quality-floor gate still missing ? the one unblocked item keeping the golden-fixtures track from DONE**
180-- anchor: `docs/GOLDEN\_REGRESSION\_FIXTURES.md:156` · tracked in: docs/GOLDEN_REGRESSION_FIXTURES.md §7 M4; docs/BACKLOG.md:60 · finder: engine:audit-doc-reconcile
181-- evidence: GOLDEN_REGRESSION_FIXTURES.md:156 (M4 row) is ?; verified tests/test_golden_quality_floor.py and eval_baselines/regression_baseline.json do not exist (ls: No such file). §9 DoD (line 197) requires M4 before the track can flip DONE. BACKLOG.md:60 calls it 'recommended-next; unblocked... the cheapest step to flip the track to DONE' (advice standing since ~06-16).
182-- why it matters: Today the golden suite is characterization-only: it catches behavior CHANGES but has no committed F1 floor or windowing fingerprint, so a deliberate re-freeze after a SERVED retune could silently lock in worse quality. It is also the sole unblocked blocker to archiving the whole fixtures project.
183-- proposed fix: Implement M4 per the doc's own recipe: seed per-slug floors from the F1 already frozen in each fx_*/expected.json score (floor = current - tolerance), record a windowing fingerprint in eval_baselines/regression_baseline.json, keep CONTROL-only, add to the golden-crossos CI matrix.
184-[Omitted long context line]
185-[Omitted long context line]
186-
187-### R1-16 [P2/L] tech-debt ? badminton-highlight-indexer
188-**Decision-snapshot regression track: decisions locked 06-25, zero code shipped since; tracker P0 row also stale**
189-- anchor: `docs/DECISION\_SNAPSHOT\_REGRESSION.md:66` · tracked in: docs/DECISION_SNAPSHOT_REGRESSION.md §7 (P1-P8) · finder: engine:audit-doc-reconcile
190-- evidence: §7 tracker (lines 63-73): P1-P8 all ?. Verified in code: grep 'build_stitch_plan' across backend/ and tests/ returns nothing (P2's pure stitch-plan seam was never factored out of backend/pipeline/stitcher/compiler.py), and tests/test_golden_fixtures.py has no stitch-plan snapshot. Minor rot: the P0 row (line 65) still shows ? '(this PR)' though the doc merged as #383 (commit 1283e18, 2026-06-25) and the header still says 'DRAFT'.
191-- why it matters: This gate exists precisely to bless genuine (non-AST-identical) refactors as quality-neutral ? i.e., exactly the class of change as the stalled #390 main.py split. Executing P1-P4 first would de-risk redoing the god-module split; leaving it a week-stale locked-decisions doc means refactors continue with no behavioral proof.
192-- proposed fix: Start §7 P1+P2 (canonical decision-snapshot schema + factor pure build_stitch_plan() out of compiler.py) as the next Claude-doable engine work; flip the P0 row to ? #383 and status to IN PROGRESS in the same PR. Ideally land P1-P4 before re-cutting the main.py split.
193-[Omitted long context line]
194-[Omitted long context line]
195-
196-### R1-17 [P2/M] tech-debt ? badminton-highlight-indexer
197-**Fetched-input scratch copies are never reclaimed anywhere (process jobs, compile jobs, /api/assets, cloud_run marker reads) ? 'OS temp reclamation' premise is false on Windows**
198-- anchor: `backend/storage/\_\_init\_\_.py:89` · tracked in: ? · finder: engine:correctness
199-[Omitted long context line]
200-- why it matters: Slow-burn disk exhaustion on exactly the appliance the product ships as; also inflates the free-space guard's false rejections.
201-- proposed fix: Wrap remote-input acquisition in a per-job scope that deletes the scratch

dir in a finally (worker already owns the job lifecycle), or route scratch under output_base(cfg)/tmp with an age-based sweep at startup; make cloud_run._read_json use TemporaryDirectory.

202-[Omitted long context line]

203-[Omitted long context line]

204-

205-### R1-18 [P2/S] critical-fix ? badminton-highlight-indexer

206-****GDriveApiStorage.fetch_to_local buffers the entire file in RAM (get_media().execute()); put() deletes the old object before the new upload succeeds****

207-- anchor: `backend/storage/gdrive\_api.py:144` · tracked in: ? · finder: engine:correctness

208-- evidence: gdrive_api.py:143-148: `data = self.\_service.files().get\_media(fileId=file\_id).execute()` then `fh.write(data)` ? the whole object is materialized as one bytes value (no MediaIoBaseDownload streaming). gdrive_api.py:163-170: put() resolves an existing file at the path and `files().delete(...)` BEFORE `files().create(...)` ? delete-then-create is not atomic.

209-- why it matters: Latent OOM + data-loss on the Drive-API storage hot path before it goes live for the reel round-trip.

210-- proposed fix: Stream downloads with googleapiclient.http.MediaIoBaseDownload in chunks; for put(), create the new file first (or upload to a temp name) and delete the old one only after success.

211-[Omitted long context line]

212-[Omitted long context line]

213-

214-### R1-19 [P2/S] critical-fix ? badminton-highlight-indexer

215-****config.json output.output_dir is silently clobbered by the CLI --output-dir DEFAULT on every server/CLI start ? the typed knob is dead, and the DB path moves with it****

216-- anchor: `backend/main.py:1992` · tracked in: ? · finder: engine:correctness

217-- evidence: main.py:1837-1841 defines `--output-dir` with default='output'; main.py:1992 unconditionally does `global\_config.output.output\_dir = resolve\_output\_dir(args.output\_dir)` ? the override fires even when the flag was never passed, so a config.json value can never take effect. models.py:274-276 documents the knob as if it's honored ('The CLI's --output-dir overrides this at startup' ? implying only when given). db_path derives from it (main.py:1995-1997).

218-- why it matters: A documented config knob that is a no-op on the main entry path, with data-location consequences (perceived data loss / split state).

219-- proposed fix: Change argparse default to None and only override when args.output_dir is not None (falling back to the config value through resolve_output_dir).

220-[Omitted long context line]

221-[Omitted long context line]

222-

223-### R1-20 [P2/S] critical-fix ? badminton-highlight-indexer

224-****upload_dir default 'output/uploads' is CWD-relative and never app-home-resolved ? packaged --app writes uploads into an arbitrary CWD (or 500s on an unwritable one) while DB/reels live under RALLY_APP_HOME****

225-- anchor: `backend/api/uploads.py:50` · tracked in: ? · finder: engine:correctness

226-- evidence: uploads.py:50 `upload\_dir = getattr(sec, 'upload\_dir', None) or 'output/uploads'` used verbatim at :89 (os.makedirs), :95, :170 ? grep confirms no resolve_output_dir/app_path is ever applied to it (only output.output_dir is resolved, main.py:1992 via app_home.py:83-100). models.py:336 default 'output/uploads'.

227-- why it matters: Breaks or misplaces the primary ingest path of the packaged app; exactly the class of CWD-vs-app-home bug P0a was built to eliminate (app_home.py:1-8).

228-- proposed fix: Resolve upload_dir through resolve_output_dir()/app_path() in _upload_cfg (relative ? app-home-rooted; absolute passes through), mirroring output_dir.

229-[Omitted long context line]

230-[Omitted long context line]

231-

232-### R1-21 [P2/M] critical-fix ? badminton-highlight-indexer

233-****Cloud Run dispatch: a bucket-poll timeout abandons a still-running GPU execution ? no cancellation, so the L4 keeps billing and its late result is never ingested****

234-- anchor: `backend/compute/cloud\_run.py:173` · tracked in: ? · finder: engine:correctness

235-- evidence: cloud_run.py:173-178: run_infer blocks on poll_until(...); execution_policy.py:183-189 raises PolicyTimeout after max_wait_sec (default 1800s). The exception propagates to main.py:1068-1070 which marks the job failed (`cloud dispatch error`) ? no code anywhere in cloud_run.py calls executions.cancel/delete (grep: no cancel API), and the late-arriving done-marker/intervals.csv are never ingested (the local job is already terminal).

3. user (2026-07-02T04:45:50.657Z)

CLAUDE.md
README.md
__pycache__
artifacts
backend
backend.db
config.json
deploy
docs
eval_baselines
indexer.db
mypy.ini
output
pyproject.toml
pytest.ini
rally_index.db
requirements.txt
ruff.toml
run_all_tests.py
run_eval.py
run_regression.py
scratch
scripts
tests
tools
training

4. user (2026-07-02T04:46:01.121Z)

```
205     ### R1-18 [P2/S] critical-fix ? badminton-highlight-indexer
206     **GDriveApiStorage.fetch_to_local buffers the entire file in RAM
(get_media().execute()); put() deletes the old object before the new upload succeeds**
207     - anchor: `backend/storage/gdrive_api.py:144` · tracked in: ? · finder:
engine:correctness
208     - evidence: gdrive_api.py:143-148: `data =
self._service.files().get_media(fileId=file_id).execute()` then `fh.write(data)` ? the whole
object is materialized as one bytes value (no MediaIoBaseDownload streaming).
gdrive_api.py:163-170: put() resolves an existing file at the path and `files().delete(...)`
BEFORE `files().create(...)` ? delete-then-create is not atomic.
209     - why it matters: Latent OOM + data-loss on the Drive-API storage hot path before it
goes live for the reel round-trip.
210     - proposed fix: Stream downloads with googleapiclient.http.MediaIoBaseDownload in
chunks; for put(), create the new file first (or upload to a temp name) and delete the old one
only after success.
211     - verifier notes: File/line accurate (line 144 is the get_media().execute() call).
Failure scenarios: (1) fetch_to_local on a multi-GB source/reel buffers the full file in RAM on
a memory-limited Cloud Run box ? OOM kill; (2) re-put() of a reel to an existing path deletes
the old file, then the resumable create fails (network/quota) ? the reel is gone with no copy at
either end. Fix refinement: for put(), prefer `files().update(fileId=existing, media_body=...)`
when a file already exists ? atomic in-place content replacement that preserves the file id and
avoids both the delete-first data-loss window and the duplicate-(name,parent) window that
create-first-then-delete would open (the file's own comment at lines 160-162 notes _resolve_id
returns the first match among duplicates). Download fix as proposed: MediaIoBaseDownload chunked
streaming.
212     - verifier reasoning: backend/storage/gdrive_api.py:143-148 ? fetch_to_local does `data
= self._service.files().get_media(fileId=file_id).execute()` then `fh.write(data)`: the whole
object is returned as a single bytes value; no MediaIoBaseDownload/chunked loop exists in the
file (only MediaFileUpload for uploads, lines 78-84). Docstring (lines 3-6, 19) states this
backend serves videos/reels on headless Cloud Run/GPU hosts, so multi-GB buffering is a genuine
OOM risk. backend/storage/gdrive_api.py:163-170 ? put() resolves `existing`, calls
`files().delete(fileId=existing).execute()`, and only then `files().create(...)` : a create
failure after the delete loses the prior object (non-atomic overwrite). git log --
backend/storage/gdrive_api.py shows only formatting/isort commits (b6214e5, f523d3a) since the
```

introducing commit 819e395 (M12, PR #338) ? not fixed. Not owner-gated: the module's owner gate (lines 14-17) covers the per-user OAuth flow, not this code's download/upload mechanics.

213

214 `### R1-19 [P2/S] critical-fix ? badminton-highlight-indexer`

215 `**config.json output.output_dir is silently clobbered by the CLI --output-dir DEFAULT on every server/CLI start ? the typed knob is dead, and the DB path moves with it**`

216 `- anchor: `backend/main.py:1992` · tracked in: ? · finder: engine:correctness`

217 `- evidence: main.py:1837-1841 defines `--output-dir` with default='output'; main.py:1992`

unconditionally does ``global_config.output.output_dir = resolve_output_dir(args.output_dir)`` ?

the override fires even when the flag was never passed, so a config.json value can never take

effect. models.py:274-276 documents the knob as if it's honored ('The CLI's --output-dir

overrides this at startup' ? implying only when given). db_path derives from it

(main.py:1995-1997).

218 `- why it matters: A documented config knob that is a no-op on the main entry path, with data-location consequences (perceived data loss / split state).`

219 `- proposed fix: Change argparse default to None and only override when args.output_dir is not None (falling back to the config value through resolve_output_dir).`

220 `- verifier notes: Path correction: the documented knob is at`

backend/config/models.py:274-276, not backend/models.py (which does not exist). Sharper failure

scenario: user sets output.output_dir: "D:/rally-data" in config.json and restarts ``indexer`

`--app``; main.py:1992 resets the dir to `resolve_output_dir("output")`, a fresh empty DB is created

there (main.py:1995-1997) so the UI shows zero videos (perceived data loss), while

run_eval.py/run_regression.py via backend/config/__init__.py:41-42 resolve the DB at D:/rally-

data ? server and eval tooling disagree on DB location. Fix scope addition: the trim path at

backend/main.py:1927-1929 also calls `resolve_output_dir(args.output_dir)` before config load and

must handle a None default (fall back to the loaded config value there too, or to "output" since

global_config isn't initialized yet at that point). Effort stays S.

221 `- verifier reasoning: Read current tree. backend/main.py:1837-1841 defines --output-dir`

with default="output". backend/main.py:1982 loads the typed config; backend/main.py:1992 then

unconditionally does `global_config.output.output_dir = resolve_output_dir(args.output_dir)`,

which fires even when the flag was never passed. `resolve_output_dir`

(backend/utils/app_home.py:83-100) never consults the config value ? with RALLY_APP_HOME unset

it returns the argparse default "output" verbatim ? so a config.json output.output_dir can never

take effect. db_path derives from the clobbered value (backend/main.py:1995-1997). The only

production entry is main() ? `create_app` (main.py:2184) ? `uvicorn.run` (main.py:2185); there is no

module-level app, so every server/CLI start hits the clobber. The knob is documented at

backend/config/models.py:274-276. Not fixed: `git log --oneline -15 -- backend/main.py` tops out

at e410136 (#413), none touching this. Split-state is concrete: backend/config/__init__.py:41-42

(`default_db_path`, used by run_eval.py/run_regression.py) builds the DB path directly from

config.json's output.output_dir with no clobber ? so with the knob set, the server and the eval

tooling resolve DIFFERENT DB locations. Not owner-gated (local config/CLI startup only).

222

223 `### R1-20 [P2/S] critical-fix ? badminton-highlight-indexer`

224 `**upload_dir default 'output/uploads' is CWD-relative and never app-home-resolved ?`

packaged --app writes uploads into an arbitrary CWD (or 500s on an unwritable one) while

DB/reels live under RALLY_APP_HOME**

225 `- anchor: `backend/api/uploads.py:50` · tracked in: ? · finder: engine:correctness`

226 `- evidence: uploads.py:50 `upload_dir = getattr(sec, 'upload_dir', None) or`

`'output/uploads'` used verbatim at :89 (os.makedirs), :95, :170 ? grep confirms no`

`resolve_output_dir/app_path` is ever applied to it (only `output.output_dir` is resolved,

main.py:1992 via app_home.py:83-100). models.py:336 default 'output/uploads'.

227 `- why it matters: Breaks or misplaces the primary ingest path of the packaged app;`

exactly the class of CWD-vs-app-home bug P0a was built to eliminate (app_home.py:1-8).

228 `- proposed fix: Resolve upload_dir through resolve_output_dir()/app_path() in`

`_upload_cfg` (relative ? app-home-rooted; absolute passes through), mirroring output_dir.

229 `- verifier notes: Sharper failure scenario: on macOS the .command runs with Terminal's`

`CWD=$HOME`, and any ``indexer --app`` shell launch (advertised in install.py:243) uses the caller's

CWD ? uploads land in e.g. `~/output/uploads` while DB/reels live under `~/Library/Application`

`Support/Khelsutra/output`; unwritable CWD ? `PermissionError` at uploads.py:89 ? 500 on

`/api/upload/init`. Caveat: Windows double-click of KhelSutra.bat coincidentally works (the .bat

lives in app_home, install.py:175, and cmd sets CWD to the .bat dir). Extra impact the finder

missed: tools/cleanup_caches.py:403 sweeps `<output-dir>/uploads` (app-home-rooted post-P0a) and

the uninstall manifest user_data (install.py:117-121) lists only app-home paths, so stray multi-

GB uploads are never swept and survive uninstall --purge. Fix as proposed is sane:

`resolve_output_dir()` in `_upload_cfg` is byte-identical with RALLY_APP_HOME unset

(app_home.py:94-95) and existing tests use absolute tmp paths (test_upload_download.py:31) that pass through.

230 - verifier reasoning: uploads.py:50 `upload\_dir = getattr(sec, "upload\_dir", None) or "output/uploads"` used verbatim at :89 os.makedirs, :93, :95, :160, :170 ? no resolver applied (repo-wide grep). main.py:1992 resolves ONLY output.output_dir via resolve_output_dir (app_home.py:83-100); models.py:336 default is relative "output/uploads"; the packaged launcher (install.py:81-99) sets RALLY_APP_HOME but never cd's, and the first-run seeded config (loader.py:261-265 LIGHT_DEFAULT_CONFIG) has no security block, so the relative default is live in every packaged install. git log -- backend/api/uploads.py (top f28af73) shows no fix. Not owner-gated (local packaging correctness).

231

232 ### R1-21 [P2/M] critical-fix ? badminton-highlight-indexer

233 **Cloud Run dispatch: a bucket-poll timeout abandons a still-running GPU execution ? no cancellation, so the L4 keeps billing and its late result is never ingested**

234 - anchor: `backend/compute/cloud\_run.py:173` · tracked in: ? · finder:
engine:correctness

5. user (2026-07-02T04:46:01.714Z)

1 # CLAUDE.md ? project memory / agent handoff

2

3 Local, AI-powered **badminton (? racket-sports) highlight indexer + rally detector**:
4 FastAPI + SQLite + ffmpeg + GPU CV/AI, with paid **Gemini** as the cloud AI. Sibling
5 repos: `sports-data-collector`, a VLC-based `rally-annotator`.

6

7 ## Read these first

8 - **docs/README.md** ? doc index. **docs/CODE_MAP.md** ? code navigation + data
9 contracts (the cold-start map; read before touching code).

10 - **Authoritative owned-model roadmap:** `docs/OWNED\_MODEL\_IMPLEMENTATION\_PLAN.md` (+
the
11 live `docs/NEXT\_STEPS.md`). These SUPERSEDE the 2026-06-07 strategy docs below where
they

12 disagree (licensing, Gemma-4 stance, base model, conversational-QA).

13 - **Strategy (post-scaffolding):** `docs/COMMERCIALIZATION.md`,
`docs/PMF\_MARKET\_RESEARCH.md`,

14 `docs/PLATFORM\_ARCHITECTURE.md`, `docs/DEFERRED\_HARDENING\_PLAN.md`. Also
`docs/ROADMAP.md`

15 (offline-segmenter narrative) and `docs/BACKLOG.md`.

16 - **Rally-detection quality track (design merged 2026-06-13, owner-gated, implementation
in progress):**

17 `docs/HOW\_RALLY\_DETECTION\_WORKS.md` (2-layer mental model) ?

`docs/MULTI\_SIGNAL\_FUSION\_PLAN.md`

18 (fuse shuttle + person + audio cues; the held-shuttle bug is the already-built
`rally\_gate.py`

19 "Gate A" **not wired into production** ? cheapest win) ? `docs/EXPERIMENT\_HARNESS.md`
(A/B + shadow

20 eval so cues land flag-gated/default-OFF with zero regression) ?

`docs/ROORA\_LABELING\_GUIDELINES.md`

21 (v8 vendor labeling spec). **docs/NEXT_STEPS.md` has the prioritized pick-up for this
track.**

22 - **History:** `docs/archives/` ? ADRs (`decisions/DECISIONS.md`), research, and

23 **checkpoints/** (latest: `2026-06-strategy-foundation/`). **Archive policy:** only
move

24 **terminal-state** (DONE/REJECTED) work; live docs stay at `docs/` top level ? and
**before

25 archiving, HONESTLY update the doc first** (verify claims vs code ? flip status + a
completion

26 note ? update inbound refs ? keep the lineage ? then `git mv`), per the checklist +
living

27 lifecycle register in **docs/DOC_STATUS.md**.

28

29 ## Current state (June 2026)

30 - **Scaffolding complete**; strategy foundation + owned-model roadmap merged. **No

31 platform/owned-model implementation has started.** A 2026-06-10 audit-driven hardening
32 sweep landed (licensing gate, detector keying, re-ingest safety, config, eval gate,

```

API
33     validation, reliability, CI + test coverage) ? see git history.
34 - **Two tracks, both owner-gated:** the **committed next PLATFORM slice = async job
model**
35     (`DEFERRED_HARDENING_PLAN.md` #16, single-tenant); the **owned-model track** starts at
a
36     greenlit Phase-0 milestone (`OWNED_MODEL_IMPLEMENTATION_PLAN.md`). **Do not start
37     implementation without explicit owner approval.**
38
39     ## Architecture in one breath
40     - **Pluggable registries** (ABC + `Registry` singleton): segmenters / providers /
validators
41         / sports / annotations ? see `backend/pipeline/segmenters/base.py`. *Future:*
42         `StorageBackend` + `ComputeBackend` registries (PLATFORM $3).
43     - `video_id` = MD5 of the file ? `backend/utils/hashing.py::compute_video_id` (one
helper).
44     - Typed config = `backend/config/` (Pydantic). DB schema evolves via the `PRAGMA
45         user_version` migration ladder in `backend/infrastructure/database.py`.
46
47     ## Committed guardrails (non-negotiable)
48     - **Paid LLMs (Gemini/OpenAI/Anthropic) = inference / serving / fallback ONLY ? never a
49         training data source** (terms/copyright). Commercial-clean licensing for EVERY
dependency ?
50         code, data, AND weights: **MIT / Apache-2.0 / BSD only** (ratified 2026-06-09, owner-
agreed).
51     - **Owned-model base:** **Qwen3-VL (Apache-2.0)** primary, InternVL3 (MIT) fallback;
**Gemma-4
52         is AMBER / bench-only** (Apache grant likely but Prohibited-Use posture unconfirmed ?
counsel
53         needed; do NOT build on it without sign-off). **Reject Gemma 3** (viral license).
**Exclude
54         minors** from the training pool; per-user training data needs a separate explicit opt-
in.
55     - **v1 = rally + highlight + history + correction loop**; **gait/biometrics strictly
future**
56         (GDPR Art. 9).
57     - **Generic data schema** ; `coach ? athlete` is an **optional overlay**, not baked in.
58     - **Compute = GCP ONLY (ADR-013, 2026-06-20):** GPU training + serving run on **GCP**
(`deploy/gcp/` ;
59         Cloud Run+GPU for serving per ADR-012). **DigitalOcean is DROPPED for compute** (DO
GPU not enabled
60         on the account + not cleanly credit-funded; Paperspace is subscription-gated). Don't
re-evaluate
61         DO/Paperspace for compute without a new ADR; the `$200` DO credits go to **non-compute
only**. The
62         `deploy/digitalocean/` scripts are kept solely as provider-agnostic Linux-GPU setup
reused by GCP.
63
64     ## Workflow conventions
65     - **Branch off `master` ; ONE PR per work item; NO PR stacking.** Don't open a PR unless
asked.
66     - Tests: `python run_all_tests.py` (offline, fully mocked). Python 3.10+ (the `trackers`
package requires ?3.10); `ffmpeg` required.
67     - Never push to the default branch directly; descriptive commit messages.
68     - **Substantial design/project work gets a tracked project doc** following
`docs/PROJECT_DOC_TEMPLATE.md` (status header · decisions-locked · progress tracker ·
definition-of-done; archived on completion). Precedents: `docs/RALLY_QUALITY_RESEARCH.md` $7,
`docs/GOLDEN_REGRESSION_FIXTURES.md`.
69     - **Launch / "bit-flip" PRs** (a default-flip that drives a tracked project to terminal
state) MUST, in the SAME PR, run the `docs/DOC_STATUS.md` $2 archival due-diligence on that
project's docs (verify-vs-code ? flip status + completion note ? update inbound refs ? keep ADR
lineage ? `git mv` to `docs/archives/` ? record in DOC_STATUS $3), and fix any internal-link
breakage the move causes. Enforced by `github/PULL_REQUEST_TEMPLATE.md` + the `link-check` CI
job (`python -m tools.check_md_links`).
70     - ?? Hand the heaviest machine-side checks (real video, the full GPU/WSL/WASB suite) to

```

```
the owner,  
71     who has the golden corpus + labelled videos; verify pure-logic paths locally.  
72
```

6. user (2026-07-02T04:46:14.197Z)

master...origin/master

7. user (2026-07-02T04:46:14.209Z)

```
1820         provider = provider_registry.get(  
1821             "gemini", api_key=api_key, model_name=config.ai_model  
1822         )  
1823         deleted = provider.cleanup_orphaned_files()  
1824         if deleted:  
1825             logger.info(  
1826                 "[STARTUP] Gemini orphan sweep reclaimed %d remote file(s).", deleted  
1827             )  
1828     except Exception as e:  
1829         logger.warning("[STARTUP] Gemini orphan sweep skipped (non-fatal): %s", e)  
1830  
1831  
1832     def main():  
1833         parser = argparse.ArgumentParser(  
1834             description="Sports Highlight Indexer Orchestrator"  
1835         )  
1836         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")  
1837         parser.add_argument(  
1838             "--output-dir",  
1839             default="output",  
1840             help="Directory where processed clips/highlights are saved",  
1841         )  
1842         parser.add_argument(  
1843             "--setup",  
1844             action="store_true",  
1845             help="Launch dependency checker and setup utility",  
1846         )  
1847         parser.add_argument(  
1848             "--debug-clip",  
1849             type=float,  
1850             help="Timestamp (in seconds) of a video clip to run detailed diagnostics on",  
1851         )  
1852         parser.add_argument(  
1853             "--server", action="store_true", help="Run the FastAPI local API server"  
1854         )  
1855         parser.add_argument(  
1856             "--app",  
1857             action="store_true",  
1858             help="Launch the app: start the local server (localhost-only) and open the "  
1859             "bundled UI in your browser. Falls back to a free port if --port is busy.",  
1860         )  
1861         parser.add_argument(  
1862             "--port", type=int, default=8000, help="Port to run the FastAPI server on"  
1863         )  
1864         parser.add_argument(  
1865             "--host",  
1866             default=None,  
1867             help="Bind address. Default 127.0.0.1 (localhost-only); 0.0.0.0 when "  
1868             "security.edge_auth_enabled. Expose on a network ONLY behind an auth gate.",  
1869         )  
1870         parser.add_argument(  
1871             "--max-frames",  
1872             type=int,  
1873             help="Limit processing to the first N sub-sampled frames for debugging",
```



```

1874     )
1875
1876     available_segmenters = list(segmenter_registry.list_available().keys())
1877     parser.add_argument(
1878         "--segmenter",
1879         choices=available_segmenters,

```

8. user (2026-07-02T04:46:21.117Z)

```

1880         help=f"Choose rally detection segmenter. Available: {available_segmenters}",
1881     )
1882     parser.add_argument(
1883         "--trim-start",
1884         type=float,
1885         help="Start time in seconds for the debug trim segment feature",
1886     )
1887     parser.add_argument(
1888         "--trim-end",
1889         type=float,
1890         help="End time in seconds for the debug trim segment feature",
1891     )
1892     parser.add_argument(
1893         "--trim-output",
1894         help="Output filename for the debug trimmed segment (saves in output-dir unless
absolute path)",
1895     )
1896
1897     args = parser.parse_args()
1898
1899     if args.setup:
1900         # Invoke the diagnostics/bootstrap script (renamed from the old root
1901         # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
1902         # build system). Resolve its path from this file so `indexer --setup`
1903         # works regardless of the caller's cwd.
1904         import subprocess
1905
1906         doctor_path = os.path.join(
1907             os.path.dirname(os.path.abspath(__file__)),
1908             "scripts",
1909             "doctor.py",
1910         )
1911         print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
1912         subprocess.run([sys.executable, doctor_path])
1913         return
1914
1915     if args.trim_start is not None and args.trim_end is not None:
1916         if not args.video_path:
1917             print("[ERROR] Please provide --video-path to extract a segment.")
1918             return
1919
1920         # Determine output path
1921         out_filename = args.trim_output or "trimmed_segment.mp4"
1922         if os.path.isabs(out_filename):
1923             out_path = out_filename
1924             out_dir = os.path.dirname(out_path)
1925             out_name = os.path.basename(out_path)
1926         else:
1927             out_dir = resolve_output_dir(
1928                 args.output_dir
1929             ) # P0a: app-home aware (CWD-identical when unset)
1930             out_path = os.path.join(out_dir, out_filename)
1931             out_name = out_filename
1932
1933         os.makedirs(out_dir, exist_ok=True)
1934         print(

```

```

1935         f"[CLI] Extracting debug segment from {args.trim_start}s to {args.trim_end}s
from {args.video_path}..."
1936     )
1937
1938     # global_config isn't initialized this early; load the typed config so the trim
1939     # clip carries the same configured stitching paddings + watermark as a real
1940     # highlight (watermark default-on).
1941     stitching = load_app_config().stitching
1942     compiler = VideoCompiler(
1943         out_dir,
1944         begin_padding=stitching.begin_padding,
1945         end_padding=stitching.end_padding,
1946         watermark=stitching.watermark,
1947     )
1948     try:
1949         res_path = compiler.compile_highlights(
1950             args.video_path, [(args.trim_start, args.trim_end)], out_name
1951         )
1952         print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
1953     except Exception as e:
1954         print(f"[ERROR] Failed to compile trimmed segment: {e}")
1955     return
1956
1957     if args.debug_clip is not None:
1958         if not args.video_path:
1959             print("[ERROR] Please provide --video-path to debug a specific clip.")
1960             return
1961         run_cli_diagnose_clip(args.video_path, args.debug_clip)
1962         return
1963
1964     # P2b (PACKAGING_PLAN.md gate 08): batteries-included first run for the friendly
1965     `--app`
1966     # launcher. Seed a truly-local, torch-free LIGHT default config (detector 'motion' ?
offline,
1967     # no API key, no GPU, no cloud bills) at the resolved app-home
1968     (default_config_path(), which
1969     # roots under RALLY_APP_HOME when the packaged shortcut pins it) BEFORE
load_app_config reads
1970     # it ? so a fresh `pip`/`uv tool install` + `indexer --app` just works. NEVER
overwrites an
1971     # existing config (a user's edits / the P2c wizard's choices are preserved). Scoped
to --app
1972     # (the packaged app mode); `--server` and the CLI single-shot are byte-identical (no
seeding).
1973     if args.app:
1974         seeded_path, seeded = write_light_default_config()
1975         if seeded:
1976             print(
1977                 f"[APP] First run: seeded a truly-local default config at {seeded_path}
"
1978                 f"(detector 'motion' ? offline, no API key or GPU needed). "
1979                 f"Use the in-app setup to switch to Gemini."
1980             )
1981
1982     # Initialize Global Config and DB
1983     global global_config, db_instance
1984     global_config = load_app_config() # returns typed AppConfig
1985     _enforce_startup_or_exit(
1986         global_config
1987     ) # M5: fail fast on an incoherent ACTIVE profile
1988
1989     # The CLI --output-dir overrides the configured output directory. It now lives
1990     # on the typed model (OutputConfig.output_dir), so every consumer ? including
1991     # /api/compile ? reads the same value instead of a write-only runtime hack.
1992     # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env

```

```

1991         # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
1992         global_config.output.output_dir = resolve_output_dir(args.output_dir)
1993         os.makedirs(global_config.output.output_dir, exist_ok=True)
1994
1995         db_path = os.path.join(
1996             global_config.output.output_dir, global_config.output.db_name
1997         )
1998
1999         # P0e: refuse a SECOND instance against the same DB BEFORE the destructive stale-job
sweep below.
2000         # The job worker is in-process, so a concurrent process opening this DB would sweep
THIS one's
2001         # in-flight jobs to 'failed'. The OS lock (held for the process lifetime; auto-
released on exit
2002         # or crash) is keyed to db_path, so a different --output-dir/DB still runs
concurrently.
2003         global _instance_lock
2004         try:
2005             _instance_lock = acquire_lock(db_path + ".lock")
2006         except OSError as exc:
2007             # The lock path isn't writable (read-only dir / perms). Don't crash and don't
falsely claim a
2008             # second instance ? warn and proceed WITHOUT the guard (a lock-infra failure
must not block a
2009             # legitimate single instance).
2010             logger.warning(
2011                 "[JOBS] Could not create the single-instance lock at %s.lock (%s);
proceeding "
2012                 "WITHOUT the second-instance guard.",
2013                 db_path,
2014                 exc,
2015             )
2016             _instance_lock = None
2017         else:
2018             if _instance_lock is None:
2019                 print(

```

9. assistant (2026-07-02T04:46:26.023Z)

Now let me look at `resolve\_output\_dir`, the config knob, and all other uses of `args.output\_dir` in main.py.

10. user (2026-07-02T04:46:27.451Z)

```

70         Returns the app-home ``.env`` ONLY when ``RALLY_APP_HOME`` is set AND that file
exists. With the
71         env UNSET this returns ``None`` so the caller keeps the historical search and does
NOT newly
72         prefer a stray CWD ``.env`` (bare ``load_dotenv()`` walks up from the caller file,
not the CWD).
73         """
74         home = app_home()
75         if (
76             str(home) == "."
77         ): # env unset (or pinned to CWD) ? historical load_dotenv() search
78             return None
79         candidate = home / ".env"
80         return candidate if candidate.exists() else None
81
82
83     def resolve_output_dir(output_dir: str) -> str:
84         """Root a RELATIVE ``output_dir`` under :func:`app_home`; pass an ABSOLUTE one
through.
85

```

```

86         With ``RALLY_APP_HOME`` UNSET the input is returned VERBATIM (the default
87         ``"output"`` when
88         empty) ? byte-identical to the pre-P0a code that stored ``--output-dir`` as-is (no
pathlib
88         normalisation of ``./x`` / separators / trailing slash). With it SET, a relative dir
like
89         ``"output"`` becomes ``<app-home>/output`` so the SQLite DB and reels live with the
rest of the
90         app's state; an absolute path ? or a Windows drive-anchored one (``C:\\x`` /
``C:x``) ? passes
91         through untouched.
92         ""
93         home = app_home()
94         if str(home) == ".": # env unset (or pinned to CWD) ? verbatim, today's behaviour
95             return output_dir or "output"
96         if not output_dir:
97             return str(home / "output")
98         if os.path.isabs(output_dir) or os.path.splitdrive(output_dir)[0]:
99             return output_dir
100        return str(home / output_dir)
101
102
103    def default_os_app_home(app_name: str = APP_DIR_NAME) -> Path:
104        """The OS-conventional per-user app-data dir (for the packaged LAUNCHER to export as
105        ``RALLY_APP_HOME`` ? NOT auto-selected by :func:`app_home`).
106
107        * Windows ? ``%APPDATA%\\<app_name>`` (``~/AppData/Roaming/<app_name>`` if unset)
108        * macOS ? ``~/Library/Application Support/<app_name>``
109        * Linux/other ? ``$XDG_DATA_HOME/<app_name>`` (``~/.local/share/<app_name>`` if
unset)

```

11. user (2026-07-02T04:46:28.415Z)

```

260        # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
261        # step makes `while t < duration: t += step` loop forever, growing memory before
any
262        # GPU work. Reject the bad config at load time instead.
263        if self.chunk_overlap >= self.chunk_duration:
264            raise ValueError(
265                f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
266                f"({self.chunk_duration}); otherwise chunking never advances."
267            )
268        return self
269
270
271    class OutputConfig(BaseModel):
272        default_highlights_name: str = "highlights.mp4"
273        db_name: str = "sports_indexer.db"
274        # Directory where the DB, highlight reels, and processed clips are written.
275        # The CLI's --output-dir overrides this at startup (see backend/main.py:main).
276        output_dir: str = "output"
277
278
279    class WatermarkConfig(BaseModel):
280        """Branding overlay burned into each highlight clip during the per-clip re-encode
281        (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
282        (under `stitching.watermark` in config.json) to turn it off. The text is fully
283        configurable. The concat stage stays stream-copy (-c copy)."""
284
285        enabled: bool = True
286        text: str = "Generated by Khelsutra.guru"
287        logo_path: str = "" # optional transparent PNG overlay; "" = no logo
288        font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family below

```

289 font: str = "Sans" # fontconfig family used when font_path is empty

12. user (2026-07-02T04:46:29.516Z)

```
1928:         args.output_dir
1992:     global_config.output.output_dir = resolve_output_dir(args.output_dir)
```

13. assistant (2026-07-02T04:46:58.810Z)

Let me check `load\_app\_config` (caching) and existing tests around this area.

14. user (2026-07-02T04:46:59.685Z)

No matches found

15. user (2026-07-02T04:47:00.085Z)

Found 16 files
tests\test_upload_download.py
tests\test_api_assets.py
tests\test_api_endpoints.py
tests\test_api_annotations.py
tests\test_workspace.py
tests\test_yolo_hybrid.py
tests\test_server_entrypoint.py
tests\test_profile_parity.py
tests\test_redact.py
tests\test_output_base.py
tests\test_flywheel.py
tests\test_generate_highlights.py
tests\test_compute_backend.py
tests\test_config.py
tests\test_app_home.py
tests\test_app_launcher.py

16. user (2026-07-02T04:47:06.912Z)

No matches found

17. user (2026-07-02T04:47:09.010Z)

```
69:         "--output-dir",
118:     """PACKAGING_PLAN.md P0a end-to-end: with RALLY_APP_HOME set and a RELATIVE --output-
dir, the
119:     SQLite DB must land under the app HOME ? not the launch CWD ? proving resolve_output_dir
is
151:         "--output-dir",
165:     "DB was NOT created under RALLY_APP_HOME ? resolve_output_dir is not wired into
main() init.\n"
176:     port clash ? is what's tested) but the SAME --output-dir."""
192:         "--output-dir",
215:     # Second instance, SAME output-dir (same DB + lock), different port ? must exit non-
zero fast.
224:         "--output-dir",
---
4:root under the configured ``output.output_dir`` (default ``"output"`` , so byte-identical to
the
5:prior literal; a custom ``--output-dir`` / ``output.output_dir`` now actually moves the
scratch).
20:     assert output_base({"output": {"output_dir": "output"}}) == "output"
23: def test_output_base_honors_custom_output_dir():
25:     cfg.output.output_dir = "/tmp/custom_out"
28:     output_base({"output": {"output_dir": "reels"}}) == "reels"
34:     assert output_base({"output": {}}) == "output" # no output_dir key
```

```

35:     assert output_base({"output": {"output_dir": ""}}) == "output" # empty -> default
109: def test_trajectory_scratch_follows_custom_output_dir(tmp_path):
110:     """A configured output_dir actually moves the trajectory scratch (the latent bug M3
fixes)."""
113:         {"indexing": {}, "output": {"output_dir": custom}}

```

18. user (2026-07-02T04:47:15.628Z)

No matches found

19. user (2026-07-02T04:47:16.567Z)

```

1     """Regression guard for the `python -m backend.main` double-module job-hang.
2
3     `python -m backend.main` runs main.py as `__main__`, so `main()` configures `__main__`'s
globals
4     (`db_instance`/`global_config`). But `job_worker`/`uploads` resolve shared state via
5     `import backend.main` ? a SEPARATE module object whose `db_instance` stays `None`. The
async-job
6     drain then crashes silently on `None.claim_due_job()` and every submitted job hangs in
`queued`
7     forever. The in-process tests (TestClient + monkeypatched `db_instance` + an inline
executor)
8     cannot see this ? only the real subprocess does. This spawns the DOCUMENTED invocation
and asserts
9     a submitted job actually LEAVES `queued`.
10    """
11
12    import json
13    import os
14    import socket
15    import subprocess
16    import sys
17    import time
18    import urllib.error
19    import urllib.request
20
21    import pytest
22
23    REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
24
25
26    def _free_port() -> int:
27        s = socket.socket()
28        s.bind(("127.0.0.1", 0))
29        port = int(s.getsockname()[1])
30        s.close()
31        return port
32
33
34    def _get(url: str, timeout: float = 5):
35        with urllib.request.urlopen(url, timeout=timeout) as r:
36            return json.loads(r.read().decode())
37
38
39    def _post(url: str, payload: dict, timeout: float = 15):
40        req = urllib.request.Request(
41            url,
42            data=json.dumps(payload).encode(),
43            headers={"content-type": "application/json"},
44        )
45        with urllib.request.urlopen(req, timeout=timeout) as r:
46            return json.loads(r.read().decode())
47
48

```

```

49     def test_python_m_backend_main_drains_submitted_jobs(tmp_path):
50         """Spawning the engine the documented way (`python -m backend.main --server`) must
actually
51         DRAIN a submitted job ? it must leave `queued`, never hang. (A dummy file fails
fast; the
52         verdict is irrelevant ? what matters is that the worker claimed and ran it.)"""
53         port = _free_port()
54         out_dir = tmp_path / "out"
55         video = tmp_path / "dummy.mp4"
56         video.write_bytes(
57             b"not-a-real-video"
58         ) # exists -> the drain claims it; processing fails fast
59         log = open(tmp_path / "server.log", "wb")
60
61         proc = subprocess.Popen(
62             [
63                 sys.executable,
64                 "-m",
65                 "backend.main",
66                 "--server",
67                 "--port",
68                 str(port),
69                 "--output-dir",
70                 str(out_dir),
71             ],
72             cwd=REPO_ROOT,
73             stdout=log,
74             stderr=subprocess.STDOUT,
75         )
76         base = f"http://127.0.0.1:{port}"
77         try:
78             deadline = time.time() + 90 # cold import of torch/cv2 can be slow on CI
79             while time.time() < deadline:
80                 if proc.poll() is not None:
81                     pytest.fail(
82                         f"server exited early (code {proc.returncode}); see {tmp_path /
'server.log'}"
83                     )
84                 try:
85                     if _get(f"{base}/api/health").get("status") == "ok":
86                         break
87                 except (urllib.error.URLError, ConnectionError, OSError):
88                     time.sleep(0.5)
89             else:
90                 pytest.fail("server never became healthy")
91
92             job_id = _post(
93                 f"{base}/api/process", {"video_path": str(video), "segmenter": "motion"}
94             )["job_id"]
95
96             status = "queued"
97             deadline = time.time() + 45
98             while time.time() < deadline:
99                 status = _get(f"{base}/api/jobs/{job_id}")["status"]
100                 if status != "queued":
101                     break
102                 time.sleep(1)
103
104             assert status != "queued", (
105                 "job stayed 'queued' ? the async drain never ran under `python -m
backend.main` "
106                 "(the __main__/backend.main double-module entrypoint bug regressed)."
107             )
108         finally:
109             proc.terminate()

```

```

110         try:
111             proc.wait(timeout=10)
112         except subprocess.TimeoutExpired:
113             proc.kill()
114         log.close()
115
116
117     def test_app_home_roots_db_under_rally_app_home(tmp_path):
118         """PACKAGING_PLAN.md P0a end-to-end: with RALLY_APP_HOME set and a RELATIVE
--output-dir, the
119         SQLite DB must land under the app HOME ? not the launch CWD ? proving
resolve_output_dir is
120         wired at the real main() init (lines that load config -> resolve output dir -> open
the DB).

```

20. user (2026-07-02T04:47:24.037Z)

```

backend\main.py:60:     load_config as load_app_config,
backend\main.py:306:# New code should import load_app_config / AppConfig from backend.config
directly.
backend\main.py:308:     cfg = load_app_config(config_path)
backend\main.py:1941:         stitching = load_app_config().stitching
backend\main.py:1967:     # roots under RALLY_APP_HOME when the packaged shortcut pins it) BEFORE
load_app_config reads
backend\main.py:1982:     global_config = load_app_config() # returns typed AppConfig

```

21. user (2026-07-02T04:47:28.956Z)

```

1     """Configuration package.
2
3     Public API:
4         from backend.config import load_config, AppConfig, Config
5
6     This package is the canonical place for all configuration access.
7     """
8
9     import os
10
11     from .loader import (
12         LIGHT_DEFAULT_CONFIG,
13         load_config,
14         save_default_config,
15         write_light_default_config,
16     )
17     from .models import ( # re-export common ones
18         AppConfig,
19         Config,
20         DeploymentConfig,
21         IndexingConfig,
22         StitchingConfig,
23         StorageConfig,
24         ValidationConfig,
25     )
26     from .presets import PROFILE_PRESETS, probe_cuda_available, suggest_profile
27     from .startup import (
28         StartupCheck,
29         StartupCheckError,
30         enforce_startup_checks,
31         run_startup_checks,
32     )
33
34
35     def default_db_path(config_path: str = "config.json") -> str:
36         """Resolve the DB path from the typed config ? the single shared helper.
37

```



```

38     Replaces the copy-pasted ``_default_db_path()`` in ``run_eval.py`` and
39     ``run_regression.py``.
40     """
41     cfg = load_config(config_path)
42     return os.path.join(cfg.output.output_dir, cfg.output.db_name)
43
44
45     __all__ = [
46         "load_config",
47         "save_default_config",
48         "write_light_default_config",
49         "LIGHT_DEFAULT_CONFIG",
50         "default_db_path",
51         "AppConfig",
52         "Config",
53         "DeploymentConfig",
54         "IndexingConfig",
55         "StitchingConfig",
56         "StorageConfig",
57         "ValidationConfig",
58         "PROFILE_PRESETS",
59         "suggest_profile",
60         "probe_cuda_available",
61         "StartupCheck",
62         "StartupCheckError",
63         "enforce_startup_checks",
64         "run_startup_checks",
65     ]
66

```

22. user (2026-07-02T04:47:37.209Z)

```

C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-2-
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\utils\paths.py-3-import os
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-4-from
typing import Any, Optional
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-5-
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-6-
backend\utils\paths.py:7:def output_base(config: Any) -> str:
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-8-
"""The configured base dir for a segmenter's LOCAL scratch/staging artifacts
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-9-
(COMPUTE_DECOUPLED_SERVING M3). Returns ``output.output_dir`` (default ``"output"``) so the
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-10-
chunk-staging / AI-handoff / temp dirs follow ``--output-dir`` / ``output.output_dir`` instead
of
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-11-
a hardcoded ``./output``. The DEFAULT is ``"output"`` ? byte-identical to the prior literal.
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-12-
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-13-
This is the LOCAL output dir, deliberately NOT the storage ``workspace_root`` (which may be a
backend\utils\paths.py-14-    ``gs://`` URI you can't ``makedirs``). Accepts the typed AppConfig
or the legacy dict-shim
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-15-
(``.get("output", {})).get("output_dir")``); any miss or empty value falls back to
``"output"``.
"""
backend\utils\paths.py-16-    try:
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-17-
out = (
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-18-
config.output.output_dir
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-19-
if not isinstance(config, dict)
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-20-

```

```

else config.get("output", {}).get("output_dir")
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-21-
)
backend\utils\paths.py-22-     except Exception:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-23-
out = None
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-24-
return out or "output"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-25-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-26-
backend\utils\paths.py-27-def safe_output_filename(name: str) -> str:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-28-
"""Validate an output filename is a BARE name (no dir part, no traversal, not absolute).
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-29-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-30-
`os.path.join(output_dir, name)` silently discards `output_dir` when `name` is absolute,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-31-
and `..` segments escape it ? so an unchecked `output_filename` is an arbitrary-write
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-32-
primitive. Returns the name unchanged when safe; raises ValueError otherwise.

```

23. user (2026-07-02T04:48:25.806Z)

```

25:from backend.utils.app_home import env_file_to_load, resolve_output_dir
73:from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
77:from backend.utils.freespace import require_free_bytes # HTTP-507 disk guard (P2, D11)
78:from backend.utils.hashing import compute_video_id # canonical file-identity hashing
79:from backend.utils.paths import (
80:    output_base,
85:from backend.utils.redact import (
88:from backend.utils.single_instance import acquire_lock # P0e: one instance per database
473:    getattr(cfg, "storage", None), local_free_path=output_base(cfg)
600:    return os.path.join(output_base(cfg), "proxies")
612:    from backend.utils.annotation_proxy import make_annotation_proxy, needs_proxy
678:    labels_dir = os.path.join(output_base(cfg), "labels")

```

24. user (2026-07-02T04:48:26.529Z)

```

117     def test_app_home_roots_db_under_rally_app_home(tmp_path):
118         """PACKAGING_PLAN.md P0a end-to-end: with RALLY_APP_HOME set and a RELATIVE
--output-dir, the
119         SQLite DB must land under the app HOME ? not the launch CWD ? proving
resolve_output_dir is
120         wired at the real main() init (lines that load config -> resolve output dir -> open
the DB).
121
122         Runs the CLI single-shot path (no --server) so it inits the DB, fails fast on a
dummy video and
123         exits ? no port/no poll. PYTHONPATH pins the subprocess to THIS tree so it exercises
the code
124         under test regardless of where the editable install points; the DB-location
assertion is itself
125         the proof the P0a wiring ran (the pre-P0a code would create ./output under the
launch CWD)."""
126         from backend.config import AppConfig
127
128         db_name = AppConfig().output.db_name
129
130         home = tmp_path / "home"
131         launch = tmp_path / "launch"
132         home.mkdir()
133         launch.mkdir()
134         video = launch / "dummy.mp4"
135         video.write_bytes(
136             b"not-a-real-video"

```

```

137     ) # exists; motion fails fast after the DB is created
138
139     env = {
140         **os.environ,
141         "RALLY_APP_HOME": str(home),
142         "PYTHONPATH": REPO_ROOT + os.pathsep + os.environ.get("PYTHONPATH", ""),
143     }
144     proc = subprocess.run(
145         [
146             sys.executable,
147             "-m",
148             "backend.main",
149             "--video-path",
150             str(video),
151             "--output-dir",
152             "output",
153             "--segmenter",
154             "motion",
155         ],
156         cwd=str(launch),
157         env=env,
158         capture_output=True,
159         text=True,
160         timeout=180,
161     )
162
163     db_under_home = home / "output" / db_name
164     assert db_under_home.exists(), (
165         "DB was NOT created under RALLY_APP_HOME ? resolve_output_dir is not wired into
main() init.\n"
166         f"stdout:\n{proc.stdout[-1500:]]}\nstderr:\n{proc.stderr[-1500:]]}"
167     )
168     assert not (launch / "output").exists(), (
169         "output/ leaked into the launch CWD (should be under the app home)"
170     )
171
172
173     def test_second_instance_on_same_db_refuses(tmp_path):
174         """PACKAGING_PLAN.md P0e: a second instance against the SAME database must refuse to
start (else
175         it would sweep the first process's in-flight jobs). Uses a DIFFERENT port (so the
lock ? not a
176         port clash ? is what's tested) but the SAME --output-dir."""
177         out_dir = str(tmp_path / "shared_out")
178         env = {
179             **os.environ,
180             "PYTHONPATH": REPO_ROOT + os.pathsep + os.environ.get("PYTHONPATH", ""),
181         }
182         port1, port2 = _free_port(), _free_port()
183         log1 = open(tmp_path / "s1.log", "wb")
184         proc1 = subprocess.Popen(
185             [
186                 sys.executable,
187                 "-m",
188                 "backend.main",
189                 "--server",
190                 "--port",
191                 str(port1),
192                 "--output-dir",
193                 out_dir,
194             ],
195             cwd=REPO_ROOT,
196             env=env,
197             stdout=log1,
198             stderr=subprocess.STDOUT,

```

```

199         )
200     try:
201         deadline = time.time() + 90 # cold torch/cv2 import
202         while time.time() < deadline:
203             if proc1.poll() is not None:
204                 pytest.fail(
205                     f"server 1 exited early (code {proc1.returncode}); see {tmp_path /
's1.log'}"
206                 )
207             try:
208                 if _get(f"http://127.0.0.1:{port1}/api/health").get("status") == "ok":
209                     break
210             except (urllib.error.URLError, ConnectionError, OSError):
211                 time.sleep(0.5)
212         else:
213             pytest.fail("server 1 never became healthy")
214
215     # Second instance, SAME output-dir (same DB + lock), different port ? must exit
non-zero fast.
216     proc2 = subprocess.run(
217         [
218             sys.executable,
219             "-m",
220             "backend.main",
221             "--server",
222             "--port",
223             str(port2),
224             "--output-dir",
225             out_dir,
226         ],
227         cwd=REPO_ROOT,
228         env=env,
229         capture_output=True,
230         text=True,
231         timeout=120,
232     )
233     assert proc2.returncode != 0, (
234         "second instance should have refused to start while the first holds the DB
lock.\n"
235         f"stdout:\n{proc2.stdout[-1500:]]}\nstderr:\n{proc2.stderr[-1500:]]}"
236     )
237     assert "Another instance is already using this database" in (
238         proc2.stdout + proc2.stderr
239     )
240     finally:
241         proc1.terminate()
242         try:
243             proc1.wait(timeout=10)
244         except subprocess.TimeoutExpired:
245             proc1.kill()
246         log1.close()

```

25. user (2026-07-02T04:48:37.258Z)

```

128: def load_config(path: str | Path | None = None) -> AppConfig:
129-     """Load configuration with the following precedence:
130-
131-     1. Built-in defaults defined in the Pydantic models.
132-     2. Values from the JSON file (if present and readable).
133-     3. (Future) environment / CLI overrides.
134-
135-     Missing file or unreadable file ? returns a fully defaulted AppConfig.
136-     Unknown keys in the file are tolerated (extra="allow" on the model).
137-     """
138-     cfg_path = Path(path) if path is not None else default_config_path()

```

```

139-
140-     file_data: dict[str, Any] = {}
141-     if cfg_path.exists():
142-         try:
143-             with open(cfg_path, "r", encoding="utf-8") as f:
144-                 file_data = json.load(f)
145-         except Exception as exc:
146-             # Non-fatal: continue with defaults + whatever we could parse.
147-             # In production we might log here.
148-             logger.warning(
149-                 f"Failed to read {cfg_path}: {exc}. Using defaults + partial data."
150-             )
151-
152-     # A config.json that is valid JSON but not an OBJECT (e.g. `[ ]`, `42`, `x`, `null`)
would
153-     # otherwise crash startup: a truthy non-mapping hits `.items()` in _unknown_key_paths,
and any
154-     # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults + warn,
per the
155-     # loader's "bad input is non-fatal" contract.
156-     if not isinstance(file_data, dict):
157-         logger.warning(
158-             "%s is not a JSON object (got %s); ignoring it and using defaults.",

```

26. user (2026-07-02T04:48:43.923Z)

```

backend\utils\app_home.py:53: def default_config_path() -> Path:
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\app_home.py-54-
""""`<app-home>/config.json` ? the config file the loader reads and `--setup` writes.
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\app_home.py-55-
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\app_home.py-56-
With `RALLY_APP_HOME` unset this is `Path("config.json")` (CWD-relative), unchanged.
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\app_home.py-57-
""""
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\app_home.py-58-
return app_home() / "config.json"
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\app_home.py-59-
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\app_home.py-60-
backend\utils\app_home.py-61- def default_env_path() -> Path:
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\app_home.py-62-
""""`<app-home>/env` ? the dotenv file the server prefers (e.g. `GEMINI_API_KEY`)."""
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\app_home.py-63-
return app_home() / ".env"
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\app_home.py-64-
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\app_home.py-65-
backend\utils\app_home.py-66- def env_file_to_load() -> Path | None:
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\app_home.py-67-
""""The `env` the server should load, or `None` meaning "use python-dotenv's DEFAULT
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\app_home.py-68-
(frame-relative walk-up) search" ? exactly the pre-P0a bare `load_dotenv()` behaviour.

```

27. user (2026-07-02T04:48:44.887Z)

20:## 0. REPO LAYOUT ? where new files go (READ BEFORE committing a new file)

28. user (2026-07-02T04:48:53.201Z)

0. REPO LAYOUT ? where new files go (READ BEFORE committing a new file)

Top-level tree and the ****placement rule**** for each kind of file. When unsure, prefer an existing ****registry / extension point**** (?) over a new top-level file.

You're adding?	Put it in?	Notes

| Backend / runtime code | `backend/<subsystem>/` |
`pipeline/{segmenters,detectors,validators}`, `providers/`, `sports/`, `annotations/`,
`infrastructure/`, `reporting/`, `stitcher/`, `utils/`, `config/`, `features/`, `api/`. Most
additions are **register a class in the relevant `?` registry**, not a new module tree. |
| **Eval LIBRARY** (importable, pure ? metrics, calibration, a scorer, a feature) |
`backend/eval/` | Pure core, no side-effects at import; CLI/I-O lives in a separate driver.
These are **imported widely** (e.g. `calibrate\_wasb` by 8 sites) ? they are library, not
scripts; do NOT move them out. |
| **Standalone run-driver / one-off entry CLI** (has `\_\_main\_\_`, NOT imported by app code) |
scripts/ | e.g. `scripts/run\_phase3\_eval.py`, `scripts/run\_process\_and\_stitch.py`. Run as
`python scripts/<name>.py`. If it needs `load\_dotenv()/`sys.path` before importing backend, add
`"scripts/<name>.py" = ["E402"]` to `ruff.toml`. |
| **Ops / maintenance tool** (not part of the served app ? cache GC, batch admin) | `tools/` |
Run as `python -m tools.<name>` (it's an importable package). e.g. `tools/cleanup\_caches.py`.
Keep the destructive decision path pure + unit-tested. |
| Training code (model fitting) | `training/` | Behind the CI-enforced import wall (ADR-008) ?
`backend.main` must not import it. Own `requirements-train.txt`. |
| A test | `tests/test\_\*.py` | pytest auto-discovers; the full-suite lane has a **70% coverage**
floor. Pure/offline + mocked (no GPU/WSL/network). |
| A doc (design, plan, living log) | `docs/` | Index it in `docs/README.md`. Only **terminal-**
state (DONE/REJECTED) work moves to `docs/archives/`. |
| A throwaway repro / debug script | `scratch/` | **Gitignored** + excluded from ruff/pytest.
Never imported by app code. |
| A model artifact (weights + manifest) | `artifacts/<name>/<ver>/` | ADR-008: **manifest**
committed (provenance), **weights gitignored**. |
| Eval baseline / leaderboard JSON | `eval\_baselines/` | Tracked (survives a clean checkout);
the no-regression gate reads it. |
| **Committed regression fixture** (video-free golden) | `eval\_baselines/fixtures/<slug>/` |
Tracked, **LF-pinned** (`.gitattributes`). Synthetic Tier-0 (or owner-cleared, de-attributed
real). Drives the `golden\_fixtures` characterization gate with **no video/GPU/DB**. See
`docs/GOLDEN\_REGRESSION\_FIXTURES.md`; mint via `python -m backend.eval.golden_fixtures --freeze-
synthetic` / `--freeze-real` (refusal-gated). |
| Pipeline run outputs (reels, trajectories, DBs, frames) | `output/` | **Gitignored**. Per-
video isolation is the tracked **PER_VIDEO_WORKSPACE.md** project (P0 helper shipped #264;
supersedes the archived `PER\_VIDEO\_OUTPUT\_LAYOUT`). |

Canonical files that stay at the repo ROOT (referenced by CI / packaging / docs ? do not
move):
`pyproject.toml` (PEP 621 packaging, hatchling; `indexer` entry point) · `run\_all\_tests.py` (CI:
`ci.yml`) ·
`run\_regression.py` (the no-regression gate CLI, cited in `EXPERIMENT\_HARNESS.md`) ·
`run\_eval.py`
(grandfathered ? imported by `tests/test\_eval\_harness.py`). The old diagnostic `setup.py` moved
to
`scripts/doctor.py` (it shadowed the build system); invoke via `python scripts/doctor.py` or
`indexer --setup`.
New standalone run-drivers go in **scripts/**, not the root.

1. PIPELINE

1A. Runtime: video file -> highlight reel

...

typed config load (built-in defaults -> config.json overrides; absent/parse-fail -> all-
defaults, never fatal)

@backend/config/loader.py::load_config ->

@backend/config/models.py::AppConfig

-> video_id = MD5(whole file)

@backend/utils/hashing.py::compute_video_id (single

shared helper, 64KB chunks; imported by main + all run_*.py ? no more per-entrypoint copies)

-> validate

@backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)

FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity ->

SceneCut -> Blur -> Relevance (? ACTUAL registration/exec order ? see

```

@backend/pipeline/validators/__init__.py)
    [validation FAIL -> persist status="failed", return early; report STILL emitted in
finally]
-> db.add_video(video_id, filepath, status, [r.dict()]) @backend/infrastructure/database.py
-> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback 'motion')
-> seg = segmenter_registry.get(seg_name)(db, global_config) ?
@backend/pipeline/segmenters/base.py::segmenter_registry (400 + available list if missing)
-> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
    [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here without
stitching]
-> finally: emit_indexer_report(...) @backend/reporting/__init__.py (? ALWAYS runs ? even
on validation failure / exception; NEVER raises; grafts response["reports"])
-> select segment ids -> [(start,end)] (+ stitching padding)
-> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
@backend/pipeline/stitcher/compiler.py
    per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy ->
output/<name>.mp4
...

Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same shape):
...

P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
-> P2 candidate "action windows" from motion/trajectory [STOP: candidates persisted via
db.add_candidate_segment]
-> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor -> provider.analyze_video(clip,
prompt) ? provider_registry
-> P4 db.add_play_segment + db.link_candidate_to_segment
...

Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate (clamp/dedup) ->
add_play_segment.
Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground truth).

```

1B. WASB shuttle substrate (P2 of wasb_hybrid)

29. assistant (2026-07-02T04:49:51.047Z)

Let me check remaining `resolve\_output\_dir` usages in main.py, the top of app_home.py and paths.py for import-cycle safety, and test_output_base.py style.

30. user (2026-07-02T04:49:52.566Z)

```

25:from backend.utils.app_home import env_file_to_load, resolve_output_dir
1927:         out_dir = resolve_output_dir(
1991:     # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
1992:     global_config.output.output_dir = resolve_output_dir(args.output_dir)

```

31. user (2026-07-02T04:49:53.878Z)

```

1     """Resolved application-data HOME for a packaged, batteries-included install.
2
3     PACKAGING_PLAN.md P0a. Today every piece of mutable state ? ``config.json``, ``.env``,
4     the SQLite DB, ``output/`` ? is resolved RELATIVE TO THE CURRENT WORKING DIRECTORY, and
5     ``scripts/doctor.py`` writes ``config.json`` to the repo root while the server reads it
from
6     the CWD. Launched from an OS app menu (CWD ? repo root) those disagree and the user
silently
7     "loses" their index. This module introduces ONE resolver every state path roots on.
8
9     Design contract ? **DEFAULT IS BYTE-IDENTICAL TO TODAY**:
10    * ``RALLY_APP_HOME`` env set ? that directory is the app home (the packaged launcher
sets it).
11    * ``RALLY_APP_HOME`` unset ? ``Path(".")`` i.e. the CWD, exactly the pre-P0a
behaviour.

```

```

12
13     The OS-default location (``%APPDATA%`` / ``~/Library/Application Support`` /
14     ``~/local/share``)
15     is computed by :func:`default_os_app_home` but is **NOT** auto-selected here ? the
16     launcher (P2)
17     calls it and exports ``RALLY_APP_HOME`` explicitly. Keeping the resolver opt-in means
18     importing
19     this module has zero behavioural effect on the repo/dev/CI/test paths (no surprise
20     writes into a
21     real user profile during a test run).
22
23     stdlib-only (os, platform, pathlib) so it is safe to import at ``backend.main`` line 1 ?
24     before
25     the heavier pydantic-backed ``backend.config`` package ? keeping the early
26     ``load_dotenv`` path light.
27
28     """
29
30     from __future__ import annotations
31
32     import os
33     import platform
34     from pathlib import Path
35
36     #: Environment variable the packaged launcher sets to pin the app-data home.
37     APP_HOME_ENV = "RALLY_APP_HOME"
38
39     #: OS-app-dir leaf used by :func:`default_os_app_home` (the product brand).
40     APP_DIR_NAME = "Khelsutra"
41
42
43     def app_home() -> Path:
44         """The resolved application-data home.
45
46         ``RALLY_APP_HOME`` (``~`` expanded) when set; otherwise ``Path(".")`` ? the CWD,
47         byte-identical
48         to the pre-P0a behaviour. Does NOT create the directory (callers create on write).
49         """
50         env = os.environ.get(APP_HOME_ENV)
51         if env and env.strip():
52             return Path(env.strip()).expanduser()
53         return Path(".")
54
55     def app_path(*parts: str | os.PathLike[str]) -> Path:
56         """A path under :func:`app_home` (e.g. ``app_path("output")``)."""
57         return app_home().joinpath(*[str(p) for p in parts])
58
59
60

```

32. user (2026-07-02T04:49:54.302Z)

```

1     """Configurable output/scratch base (COMPUTE_DECOUPLED_SERVING M3).
2
3     The segmenters' chunk-staging / AI-handoff / temp dirs no longer hardcode ``./output`` ?
4     they
5     root under the configured ``output.output_dir`` (default ``"output"`` , so byte-identical
6     to the
7     prior literal; a custom ``--output-dir`` / ``output.output_dir`` now actually moves the
8     scratch).
9     """
10
11     import os
12     from unittest.mock import MagicMock, patch
13
14     from backend.config.models import AppConfig

```



```

12 from backend.pipeline.segmenters.gemini import _chunk_workdir
13 from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
14 from backend.utils.paths import output_base
15
16
17 # ----- output_base() helper
18 def test_output_base_default_is_output():
19     assert output_base(AppConfig()) == "output" # typed config, built-in default
20     assert output_base({"output": {"output_dir": "output"}}) == "output"
21
22
23 def test_output_base_honors_custom_output_dir():
24     cfg = AppConfig()
25     cfg.output.output_dir = "/tmp/custom_out"
26     assert output_base(cfg) == "/tmp/custom_out"
27     assert (
28         output_base({"output": {"output_dir": "reels"}}) == "reels"
29     ) # legacy dict-shim
30
31
32 def test_output_base_falls_back_to_output_on_missing_or_empty():
33     assert output_base({}) == "output" # no output section
34     assert output_base({"output": {}}) == "output" # no output_dir key
35     assert output_base({"output": {"output_dir": ""}}) == "output" # empty -> default
36     assert output_base(None) == "output" # defensive: bad config
37
38
39 # ----- _chunk_workdir base
40 def test_chunk_workdir_default_base_is_byte_identical():
41     # The no-arg call (used by tests / any legacy caller) is unchanged.
42     assert _chunk_workdir("vidA") == os.path.join("output", "chunks", "vidA")
43
44
45 def test_chunk_workdir_honors_custom_base():
46     assert _chunk_workdir("vidA", "/tmp/out") == os.path.join(
47         "/tmp/out", "chunks", "vidA"
48     )
49
50
51 # ----- segmenter call-site
52 def _mock_runner():
53     runner = MagicMock()
54     runner.healthcheck.return_value = (True, "env ok")
55     runner.run_predict.return_value = "out.csv"
56     runner.parse_trajectory_csv.return_value = [MagicMock(visible=True)]
57     runner.trajectory_to_action_windows.return_value = [
58         {
59             "start": 1.0,
60             "end": 4.0,
61             "active_frame_count": 30,
62             "peak_velocity": 0.4,
63             "mean_velocity": 0.2,
64             "track_id_count": 1,
65             "chunk_index": 0,
66         }
67     ]
68     return runner
69
70
71 def _run_wasb_capture_out_dir(config):
72     """Run wasb_hybrid through the standard mocked pipeline; return the out_dir passed
73     to
74     run_predict (Phase 2 trajectory dir)."""
75     runner = _mock_runner()
76     db = MagicMock()

```

```

76         db.add_candidate_segment.return_value = 55
77         db.add_play_segment.return_value = 200
78         provider = MagicMock()
79         provider.analyze_video.return_value = (
80             [{"start_time": 0.5, "end_time": 3.0, "shuttle_exchange_count": 4}],
81             {},
82         )
83         with patch.object(
84             WasbHybridSegmenter,
85             "_build_runner",
86             return_value=(runner, {"weights_path": "w"}),
87         ), patch.object(
88             WasbHybridSegmenter, "_probe_fps_width", return_value=(30.0, 1280.0)
89         ), patch(
90             "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
91             return_value=30.0,
92         ), patch(
93             "backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
94             return_value=True,
95         ), patch(
96             "backend.pipeline.segmenters.trajectory_hybrid.provider_registry"
97         ) as preg, patch("os.makedirs"), patch("os.environ.get", return_value="key"):
98             preg.get.return_value = provider
99             seg = WasbHybridSegmenter(db, config)
100             seg.process_video("v1", "clip.mp4")
101             return runner.run_predict.call_args[0][1] # 2nd positional arg = out_dir
102
103
104     def test_trajectory_scratch_default_is_literal_output():
105         """Default config: out_dir is `output/wasb` ? byte-identical to the prior
106         hardcoded."""
107         assert _run_wasb_capture_out_dir({"indexing": {}}) == os.path.join("output", "wasb")
108
109     def test_trajectory_scratch_follows_custom_output_dir(tmp_path):
110         """A configured output_dir actually moves the trajectory scratch (the latent bug M3
111         fixes)."""
112         custom = str(tmp_path / "custom_out")
113         out_dir = _run_wasb_capture_out_dir(
114             {"indexing": {}, "output": {"output_dir": custom}}
115         )
116         assert out_dir == os.path.join(custom, "wasb")

```

33. assistant (2026-07-02T04:50:07.661Z)

Now let me check pytest.ini, ruff/mypy configs, and recent commit style.

34. user (2026-07-02T04:50:08.627Z)

```

1     [pytest]
2     # When `pytest` is invoked without an explicit path, only collect from tests/.
3     # scratch/ holds debugging scripts that hit real APIs (Gemini upload, etc.)
4     # and should never run as part of the regular test suite.
5     testpaths = tests
6

```

35. user (2026-07-02T04:50:09.784Z)

e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)
aa915a7 deploy/gcp: centralize project + bucket names in one sourced buckets.env (#410)
52d97b5 P4: POST /api/assets ? store a video into the user's chosen medium (#409)

```
7ea6ea9 P3: /api/capabilities reports usable storage backends (#408)
f28af73 P2: engine free-space guard (HTTP 507 + WSL frame-stage BLOCK) (#407)
759a67f feat(api): frame-preserving annotation proxy for /api/source (D4) (#406)
5a5c4fa feat(api): P8a ? GET /api/source + POST /api/annotations (annotate-in-UI backend) (#405)
4a7bfbe docs: record the launch NO-GO decision (owner, 2026-06-30) (#404)
6e973af docs: fix stale rally-cluster claims (no archival ? cluster still gated) (#403)
af1ccbe chore(#399): launch-PR process hardening ? link-check CI, PR template, doc cross-links
(#402)
[mypy]
```

Lenient, GREEN baseline (quality sweep AI-4): the gate protects the clean modules and pins an explicit quarantine list below ? not a typing crusade. Ratchet: when you touch a quarantined module, try removing its entry.

```
ignore_missing_imports = True
backend/ is a namespace package (no root __init__.py) ? required so mypy
maps files to backend.* module names instead of duplicating top-level names.
explicit_package_bases = True
```

--- Quarantine (known-loose modules, each with a reason) ---

Optional module globals (db_instance/global_config set by create_app() / main()). Route handlers use request.app.state exclusively; helpers take explicit params. backend.api.job_worker lifted from quarantine (now takes explicit config/db params).

```
[mypy-backend.main]
ignore_errors = True
```

Worker dispatcher: composes the SAME validator/segmenter/report building blocks a backend.main, which are typed `config: dict` but receive the AppConfig migration shim at runtime (the .get() compat layer). Shares backend.main's config-shim quarantine until the typed-config migration removes the dict?AppConfig duality everywhere.

```
[mypy-backend.worker.__main__]
ignore_errors = True
```

yolo_hybrid: the fusion-P1 extraction moved its loose torchvision/cv2 inline code to detectors/person_detector.py (which IS type-clean, no quarantine). What remains here is pre-existing orchestration debt unrelated to that move (implicit-Optional process_video defaults, the Success|Failure ABC-override mismatch, a Future[...] annotation) ? ratchet those out in a focused typing pass, not here. (wasb_hybrid/tracknet_hybrid left the quarantine with the AI-7 collapse.)

```
[mypy-backend.pipeline.segmenters.yolo_hybrid]
ignore_errors = True
```

```
[mypy-backend.pipeline.segmenters.gemini]
ignore_errors = True
```

Registry builds cls(None, {}) probes; typing it honestly needs a Protocol.

```
[mypy-backend.pipeline.segmenters.base]
ignore_errors = True
```

Lint-only lane (no `ruff format` enforcement ? a whole-repo reformat would bury blame history for zero behavior value).

Selected: pyflakes + pycodestyle errors (ruff defaults) + pycodestyle warnings (W) + bugbear (B) + isort (I ? consistent import ordering).

target-version = "py38"

scratch/ = throwaway debugging scripts that may hit real APIs; it is also excluded from pytest collection (see pytest.ini).

extend-exclude = ["scratch"]

[lint]
extend-select = ["W", "B", "I"]

[lint.per-file-ignores]

Entrypoint scripts deliberately run setup (load_dotenv, logging config, sys.path) before importing backend modules ? import order is load-bearing.

"backend/main.py" = ["E402"]
"scripts/run_phase3_eval.py" = ["E402"]
"scripts/run_process_and_stitch.py" = ["E402"]

36. assistant (2026-07-02T04:50:17.481Z)

I have everything I need. Setting up the isolated worktree per the safety protocol.

37. user (2026-07-02T04:50:22.151Z)

Preparing worktree (new branch 'fix/output-dir-config-clobber')
branch 'fix/output-dir-config-clobber' set up to track 'origin/master'.
HEAD is now at e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
fix/output-dir-config-clobber
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)

38. assistant (2026-07-02T04:50:32.242Z)

Worktree is on my branch at origin/master. Now let me read the worktree copies of the files I'll edit.

39. user (2026-07-02T04:50:33.480Z)

```
1      """Path-safety helpers for the API boundary (untrusted request fields)."""
2
3      import os
4      from typing import Any, Optional
5
6
7      def output_base(config: Any) -> str:
8          """The configured base dir for a segmenter's LOCAL scratch/staging artifacts
9             (COMPUTE_DECOUPLED_SERVING M3). Returns ``output.output_dir`` (default ``"output"``)
so the
10         chunk-staging / AI-handoff / temp dirs follow ``--output-dir`` /
``output.output_dir`` instead of
11         a hardcoded ``./output``. The DEFAULT is ``"output"`` ? byte-identical to the prior
literal.
12
13         This is the LOCAL output dir, deliberately NOT the storage ``workspace_root`` (which
may be a
14         ``gs://`` URI you can't ``mkdirs``). Accepts the typed AppConfig or the legacy
dict-shim
15         (``.get("output", {})).get("output_dir")``); any miss or empty value falls back to
``"output"``."""
```

```

16         try:
17             out = (
18                 config.output.output_dir
19                 if not isinstance(config, dict)
20                 else config.get("output", {}).get("output_dir")
21             )
22         except Exception:
23             out = None
24         return out or "output"
25
26
27     def safe_output_filename(name: str) -> str:
28         """Validate an output filename is a BARE name (no dir part, no traversal, not
absolute).
29
30         `os.path.join(output_dir, name)` silently discards `output_dir` when `name` is
absolute,
31         and `..` segments escape it ? so an unchecked `output_filename` is an arbitrary-
write
32         primitive. Returns the name unchanged when safe; raises ValueError otherwise.
33         """
34         if not name or not name.strip():
35             raise ValueError("output_filename must not be empty")
36         if "\x00" in name:
37             raise ValueError("output_filename must not contain a null byte")
38         if "/" in name or "\\" in name:
39             raise ValueError(f"output_filename must not contain a path separator: {name!r}")
40         if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
41             raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
42         return name
43
44
45     def resolve_under_root(path: str, root: str) -> str:
46         """Return the real path, asserting it is contained under `root` (raises ValueError
if not)."""
47         rp = os.path.realpath(path)
48         rr = os.path.realpath(root)
49         try:
50             contained = os.path.commonpath([rp, rr]) == rr
51         except ValueError: # different drives on Windows
52             contained = False
53         if not contained:
54             raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
55         return rp
56
57
58     def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
59         """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
60
61         With no allowed_root (default, single-user local mode) only the null-byte check
applies,
62         so the local 'point it at any of my videos' workflow is preserved. Setting an
63         allowed_root (a future hosted/tenant deployment) enforces containment.
64         """
65         if "\x00" in path:
66             raise ValueError("video_path must not contain a null byte")
67         if allowed_root:
68             resolve_under_root(path, allowed_root)
69

```

40. user (2026-07-02T04:50:41.457Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-output-dir-config/backend/utils/paths.py has been updated successfully. (file state is current in your

context ? no need to Read it back)

41. user (2026-07-02T04:50:51.415Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-output-dir-config/backend/utils/paths.py has been updated successfully. (file state is current in your context ? no need to Read it back)

42. user (2026-07-02T04:50:59.733Z)

```
75         ffprobe_bin,
76     )
77     from backend.utils.freespace import require_free_bytes # HTTP-507 disk guard (P2, D11)
78     from backend.utils.hashing import compute_video_id # canonical file-identity hashing
79     from backend.utils.paths import (
80         output_base,
81         resolve_under_root,
82         safe_output_filename,
83         validate_input_path,
84     )
85     from backend.utils.redact import (
86         redact_secrets, # P0b: mask secret-shaped fields in /api/config
87     )
88     from backend.utils.single_instance import acquire_lock # P0e: one instance per database
89
```

43. user (2026-07-02T04:51:06.683Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-output-dir-config/backend/main.py has been updated successfully. (file state is current in your context ? no need to Read it back)

44. user (2026-07-02T04:51:13.101Z)

```
1835         description="Sports Highlight Indexer Orchestrator"
1836     )
1837     parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
1838     parser.add_argument(
1839         "--output-dir",
1840         default="output",
1841         help="Directory where processed clips/highlights are saved",
1842     )
1843     parser.add_argument(
1844         "--setup",
1845         action="store_true",
1846         help="Launch dependency checker and setup utility",
1847     )
1848
```

45. user (2026-07-02T04:51:20.946Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-output-dir-config/backend/main.py has been updated successfully. (file state is current in your context ? no need to Read it back)

46. assistant (2026-07-02T04:51:27.065Z)

Now the trim path and the main init site.

47. user (2026-07-02T04:51:28.470Z)

```
1916
1917     if args.trim_start is not None and args.trim_end is not None:
1918         if not args.video_path:
1919             print("[ERROR] Please provide --video-path to extract a segment.")
1920
```

```

1920         return
1921
1922     # Determine output path
1923     out_filename = args.trim_output or "trimmed_segment.mp4"
1924     if os.path.isabs(out_filename):
1925         out_path = out_filename
1926         out_dir = os.path.dirname(out_path)
1927         out_name = os.path.basename(out_path)
1928     else:
1929         out_dir = resolve_output_dir(
1930             args.output_dir
1931         ) # P0a: app-home aware (CWD-identical when unset)
1932         out_path = os.path.join(out_dir, out_filename)
1933         out_name = out_filename
1934
1935     os.makedirs(out_dir, exist_ok=True)
1936     print(
1937         f"[CLI] Extracting debug segment from {args.trim_start}s to {args.trim_end}s
from {args.video_path}..."
1938     )
1939
1940     # global_config isn't initialized this early; load the typed config so the trim
1941     # clip carries the same configured stitching paddings + watermark as a real
1942     # highlight (watermark default-on).
1943     stitching = load_app_config().stitching
1944     compiler = VideoCompiler(
1945         out_dir,
1946         begin_padding=stitching.begin_padding,
1947         end_padding=stitching.end_padding,
1948         watermark=stitching.watermark,
1949     )
1950     try:
1951         res_path = compiler.compile_highlights(
1952             args.video_path, [(args.trim_start, args.trim_end)], out_name
1953         )
1954         print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
1955     except Exception as e:
1956         print(f"[ERROR] Failed to compile trimmed segment: {e}")
1957     return
1958
1959 if args.debug_clip is not None:
1960     if not args.video_path:
1961         print("[ERROR] Please provide --video-path to debug a specific clip.")
1962         return
1963     run_cli_diagnose_clip(args.video_path, args.debug_clip)
1964     return
1965
1966 # P2b (PACKAGING_PLAN.md gate 08): batteries-included first run for the friendly
`--app`
1967 # launcher. Seed a truly-local, torch-free LIGHT default config (detector 'motion' ?
offline,
1968 # no API key, no GPU, no cloud bills) at the resolved app-home
(default_config_path(), which
1969 # roots under RALLY_APP_HOME when the packaged shortcut pins it) BEFORE
load_app_config reads
1970 # it ? so a fresh `pip`/`uv tool install` + `indexer --app` just works. NEVER
overwrites an
1971 # existing config (a user's edits / the P2c wizard's choices are preserved). Scoped
to --app
1972 # (the packaged app mode); `--server` and the CLI single-shot are byte-identical (no
seeding).
1973 if args.app:
1974     seeded_path, seeded = write_light_default_config()
1975     if seeded:
1976         print(

```

```

1977         f"[APP] First run: seeded a truly-local default config at {seeded_path}"
1978     "
1979         f"(detector 'motion' ? offline, no API key or GPU needed). "
1980         f"Use the in-app setup to switch to Gemini."
1981     )
1982     # Initialize Global Config and DB
1983     global global_config, db_instance
1984     global_config = load_app_config() # returns typed AppConfig
1985     _enforce_startup_or_exit(
1986         global_config
1987     ) # M5: fail fast on an incoherent ACTIVE profile
1988
1989     # The CLI --output-dir overrides the configured output directory. It now lives
1990     # on the typed model (OutputConfig.output_dir), so every consumer ? including
1991     # /api/compile ? reads the same value instead of a write-only runtime hack.
1992     # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
1993     # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
1994     global_config.output.output_dir = resolve_output_dir(args.output_dir)
1995     os.makedirs(global_config.output.output_dir, exist_ok=True)
1996
1997     db_path = os.path.join(
1998         global_config.output.output_dir, global_config.output.db_name
1999     )
2000

```

48. user (2026-07-02T04:51:37.501Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-output-dir-config/backend/main.py has been updated successfully. (file state is current in your context ? no need to Read it back)

49. user (2026-07-02T04:51:49.163Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-output-dir-config/backend/main.py has been updated successfully. (file state is current in your context ? no need to Read it back)

50. user (2026-07-02T04:51:57.799Z)

```
25:from backend.utils.app_home import env_file_to_load, resolve_output_dir
```

51. user (2026-07-02T04:52:03.788Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-output-dir-config/backend/main.py has been updated successfully. (file state is current in your context ? no need to Read it back)

52. assistant (2026-07-02T04:52:12.872Z)

Now the doc comment in models.py that the audit flagged as misleading.

53. user (2026-07-02T04:52:14.147Z)

```

271     class OutputConfig(BaseModel):
272         default_highlights_name: str = "highlights.mp4"
273         db_name: str = "sports_indexer.db"
274         # Directory where the DB, highlight reels, and processed clips are written.
275         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
276         output_dir: str = "output"
277
278

```


54. user (2026-07-02T04:52:25.298Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-output-dir-config/backend/config/models.py has been updated successfully. (file state is current in your context ? no need to Read it back)

55. assistant (2026-07-02T04:52:31.104Z)

Now the tests. Let me check test_app_home.py's env-handling style for consistency.

56. user (2026-07-02T04:52:31.600Z)

```
1      """Tests for the resolved application-data HOME (PACKAGING_PLAN.md P0a).
2
3      The load-bearing guarantee: with ``RALLY_APP_HOME`` UNSET, every resolver is byte-
identical to the
4      pre-P0a CWD-relative behaviour; with it SET, config/.env/output/DB all root under that
one home.
5      """
6
7      import os
8      from pathlib import Path
9
10     import pytest
11
12     from backend.config import loader
13     from backend.utils import app_home as ah
14
15
16     @pytest.fixture(autouse=True)
17     def _clear_app_home_env(monkeypatch):
18         """Each test starts with RALLY_APP_HOME unset unless it sets it explicitly."""
19         monkeypatch.delenv(ah.APP_HOME_ENV, raising=False)
20
21
22     # ---- default (env unset) is byte-identical to today ----
23
24
25     def test_app_home_defaults_to_cwd():
26         assert ah.app_home() == Path(".")
27
28
29     def test_default_config_path_unset_is_cwd_config_json():
30         # Pre-P0a this was the module constant Path("config.json").
31         assert ah.default_config_path() == Path("config.json")
32
33
34     def test_default_env_path_unset_is_cwd_dotenv():
35         assert ah.default_env_path() == Path(".env")
36
37
38     def test_resolve_output_dir_unset_is_verbatim_for_relative():
39         # env unset ? returned VERBATIM: no pathlib normalisation of ./ , separators, or
trailing slash
40         # (the pre-P0a code stored --output-dir as-is, and that string is echoed by
/api/config + reports).
41         for v in ("output", "custom", "./output", "out/clips", "output/"):
42             assert ah.resolve_output_dir(v) == v
43             assert ah.resolve_output_dir("") == "output" # empty ? the historical default
44
45
46     def test_env_file_to_load_unset_returns_none(monkeypatch, tmp_path):
47         # Even with a .env sitting in the CWD, the unset path must NOT prefer it ? bare
load_dotenv()
```

```

48         # walks up from the caller FILE, not the CWD, so None preserves that exact
behaviour.
49         monkeypatch.chdir(tmp_path)
50         (tmp_path / ".env").write_text("X=1\n", encoding="utf-8")
51         assert ah.env_file_to_load() is None
52
53
54     # ---- RALLY_APP_HOME set ? everything roots there ----
55
56
57     def test_app_home_honors_env(monkeypatch, tmp_path):
58         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
59         assert ah.app_home() == tmp_path
60
61
62     def test_app_home_expands_user(monkeypatch):
63         monkeypatch.setenv(ah.APP_HOME_ENV, "~/khelesutra-data")
64         assert ah.app_home() == Path.home() / "khelesutra-data"
65
66
67     def test_app_home_ignores_blank_env(monkeypatch):
68         monkeypatch.setenv(ah.APP_HOME_ENV, " ")
69         assert ah.app_home() == Path(".")
70
71
72     def test_config_and_env_paths_root_under_home(monkeypatch, tmp_path):
73         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
74         assert ah.default_config_path() == tmp_path / "config.json"
75         assert ah.default_env_path() == tmp_path / ".env"
76         assert ah.app_path("output", "x.db") == tmp_path / "output" / "x.db"
77
78
79     def test_resolve_output_dir_set(monkeypatch, tmp_path):
80         monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))

```

57. assistant (2026-07-02T04:52:48.573Z)

Now I'll write the new test file in the worktree.

58. assistant (2026-07-02T04:53:15.888Z)

```

"""Startup output-dir precedence: CLI --output-dir > config.json output.output_dir > "output".

```

Regression guard for the audit finding where the argparse default ``"output"`` was applied UNCONDITIONALLY at ``backend.main::main`` init, silently clobbering the documented ``output.output\_dir`` knob (backend/config/models.py::OutputConfig) ? and moving the SQLite DB with it ? on every server/CLI start. The fix: argparse default is ``None`` and ``backend.utils.paths.resolve\_startup\_output\_dir`` only lets the CLI win when the flag was actually passed.

Two layers:

```

* pure unit tests of ``resolve_startup_output_dir`` (precedence + P0a app-home rooting);
* subprocess wiring tests spawning the real CLI single-shot path (no server, no port ?
  the dummy video fails fast AFTER the DB is created) asserting where the DB lands.
"""

```

```

import json
import os
import subprocess
import sys

```

```

import pytest

```

```

from backend.config.models import AppConfig
from backend.utils import app_home as ah

```

```

from backend.utils.paths import resolve_startup_output_dir

REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

@pytest.fixture(autouse=True)
def _clear_app_home_env(monkeypatch):
    """Each test starts with RALLY_APP_HOME unset unless it sets it explicitly."""
    monkeypatch.delenv(ah.APP_HOME_ENV, raising=False)

```

----- unit: precedence

```

def test_neither_set_is_byte_identical_default():
    # No CLI flag, all-defaults config ? the historical literal "output".
    assert resolve_startup_output_dir(None, AppConfig()) == "output"

def test_config_only_is_honored():
    """The exact audit bug: a config-set output_dir must survive startup without the flag."""
    cfg = AppConfig()
    cfg.output.output_dir = "D:/rally-data"
    assert resolve_startup_output_dir(None, cfg) == "D:/rally-data"

def test_config_only_relative_is_verbatim_when_env_unset():
    cfg = AppConfig()
    cfg.output.output_dir = "reels"
    assert resolve_startup_output_dir(None, cfg) == "reels"

def test_cli_wins_over_config():
    cfg = AppConfig()
    cfg.output.output_dir = "D:/rally-data"
    assert resolve_startup_output_dir("C:/cli-dir", cfg) == "C:/cli-dir"

def test_empty_config_value_falls_back_to_output():
    cfg = AppConfig()
    cfg.output.output_dir = ""
    assert resolve_startup_output_dir(None, cfg) == "output"

def test_empty_cli_string_falls_back_to_output():
    # `--output-dir ""` is "passed" but empty ? the historical default, not a crash.
    cfg = AppConfig()
    cfg.output.output_dir = "D:/rally-data" # CLI (even empty) still wins over config
    assert resolve_startup_output_dir("", cfg) == "output"

def test_legacy_dict_shim_config_works():
    assert (
        resolve_startup_output_dir(None, {"output": {"output_dir": "shim_out"}})
        == "shim_out"
    )

```

----- unit: P0a app-home rooting

```

def test_config_relative_roots_under_app_home(monkeypatch, tmp_path):
    monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))

```

```

cfg = AppConfig()
cfg.output.output_dir = "reels"
assert resolve_startup_output_dir(None, cfg) == str(tmp_path / "reels")

def test_cli_relative_roots_under_app_home(monkeypatch, tmp_path):
    monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path))
    assert resolve_startup_output_dir("custom", AppConfig()) == str(
        tmp_path / "custom"
    )

def test_config_absolute_passes_through_app_home(monkeypatch, tmp_path):
    monkeypatch.setenv(ah.APP_HOME_ENV, str(tmp_path / "home"))
    absolute = str(tmp_path / "elsewhere")
    cfg = AppConfig()
    cfg.output.output_dir = absolute
    assert resolve_startup_output_dir(None, cfg) == absolute

```

----- wiring: the real main() init

```

def _run_single_shot(launch_dir, extra_args=()):
    """Spawn the CLI single-shot path (no server): inits config + DB, fails fast on the
    dummy video, exits. Mirrors tests/test_server_entrypoint.py's P0a wiring test."""
    video = launch_dir / "dummy.mp4"
    video.write_bytes(b"not-a-real-video") # exists; motion fails fast after DB init
    env = {
        **os.environ,
        "PYTHONPATH": REPO_ROOT + os.pathsep + os.environ.get("PYTHONPATH", ""),
    }
    env.pop(ah.APP_HOME_ENV, None) # CWD-relative resolution, like a bare dev run
    return subprocess.run(
        [
            sys.executable,
            "-m",
            "backend.main",
            "--video-path",
            str(video),
            "--segmenter",
            "motion",
            *extra_args,
        ],
        cwd=str(launch_dir),
        env=env,
        capture_output=True,
        text=True,
        timeout=180,
    )

def test_main_honors_config_output_dir_without_cli_flag(tmp_path):
    """End-to-end proof of the fix: output.output_dir set ONLY in config.json moves the DB
    there ? and no clobbered ./output is created in the launch CWD."""
    db_name = AppConfig().output.db_name
    launch = tmp_path / "launch"
    cfg_dir = tmp_path / "from_config"
    launch.mkdir()
    (launch / "config.json").write_text(
        json.dumps({"output": {"output_dir": str(cfg_dir)}}), encoding="utf-8"
    )

    proc = _run_single_shot(launch)

```

```

assert (cfg_dir / db_name).exists(), (
    "DB was NOT created under config.json's output.output_dir ? the CLI default "
    "clobbered the configured knob again.\n"
    f"stdout:\n{proc.stdout[-1500:]} \nstderr:\n{proc.stderr[-1500:]}")
)
assert not (launch / "output").exists(), (
    "output/ appeared in the launch CWD ? the argparse default overrode the config"
)

def test_main_cli_output_dir_beats_config(tmp_path):
    """An explicit --output-dir still wins over a conflicting config.json value."""
    db_name = AppConfig().output.db_name
    launch = tmp_path / "launch"
    cfg_dir = tmp_path / "from_config"
    cli_dir = tmp_path / "from_cli"
    launch.mkdir()
    (launch / "config.json").write_text(
        json.dumps({"output": {"output_dir": str(cfg_dir)}}), encoding="utf-8"
    )

    proc = _run_single_shot(launch, extra_args="--output-dir", str(cli_dir)))

    assert (cli_dir / db_name).exists(), (
        "DB was NOT created under the explicit --output-dir.\n"
        f"stdout:\n{proc.stdout[-1500:]} \nstderr:\n{proc.stderr[-1500:]}")
    )
    assert not (cfg_dir / db_name).exists(), (
        "DB leaked into config.json's output_dir despite an explicit --output-dir"
    )

```

59. user (2026-07-02T04:53:15.925Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-output-dir-config/tests/test_startup_output_dir.py (file state is current in your context ? no need to Read it back)

60. assistant (2026-07-02T04:53:24.316Z)

Now let's run the targeted tests. First check the Python env in the worktree.

61. user (2026-07-02T04:53:26.908Z)

Python 3.13.13
deps ok

62. user (2026-07-02T04:53:34.170Z)

```

..... [100%]
10 passed, 2 deselected in 0.40s

```

63. assistant (2026-07-02T04:53:40.622Z)

Unit tests pass. Now the two subprocess wiring tests (heavier ? real `python -m backend.main` runs).

64. user (2026-07-02T04:53:52.513Z)

```

.. [100%]
2 passed, 10 deselected in 9.00s

```

65. assistant (2026-07-02T04:53:59.574Z)

Wiring tests pass too. Now run the neighboring test files that cover the touched surfaces (output_base, app_home, the entrypoint tests that pass `--output-dir` explicitly, config).

66. user (2026-07-02T04:54:07.085Z)

```
..... [100%]  
53 passed in 2.93s
```

67. user (2026-07-02T04:54:27.574Z)

```
... [100%]  
3 passed in 13.23s
```

68. user (2026-07-02T04:55:13.340Z)

All checks passed!
backend\pipeline\validators\base.py:44: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers__init__.py:9: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 3 source files

69. assistant (2026-07-02T04:55:20.391Z)

Lint and types are clean. Let me confirm what CI actually runs (mypy scope) to make sure I match it.

70. user (2026-07-02T04:55:23.457Z)

```
.github/workflows/ci.yml:16: # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle +  
bugbear.  
.github/workflows/ci.yml:17: # Also the repo-wide syntax guard ? ruff reports invalid-syntax on  
every  
.github/workflows/ci.yml:29: - name: Install ruff  
.github/workflows/ci.yml:30:   run: python -m pip install ruff  
.github/workflows/ci.yml:32:   run: ruff check . --output-format github  
.github/workflows/ci.yml:65: # Type gate (mypy.ini at repo root): lenient green baseline with  
an explicit  
.github/workflows/ci.yml:67: # up. Typed core deps are installed so mypy sees their real  
signatures  
.github/workflows/ci.yml:78: - name: Install mypy + typed core deps  
.github/workflows/ci.yml:79:   run: python -m pip install mypy pydantic fastapi uvicorn  
python-dotenv google-genai numpy  
.github/workflows/ci.yml:81:   run: mypy backend training  
.github/workflows/ci.yml:135:   # (pytest, pytest-cov, httpx for the fastapi TestClient,  
ruff, mypy).  
.github/workflows/nightly.yml:43: # Lint gate (ruff.toml at repo root) ? mirrors ci.yml::lint.
```

71. user (2026-07-02T04:55:37.258Z)

All checks passed!
backend\storage\gcs.py:31: error: Module "google.cloud" has no attribute "storage" [attr-
defined]
Found 1 error in 1 file (checked 141 source files)

72. user (2026-07-02T04:55:48.254Z)

Success: no issues found in 1 source file

73. user (2026-07-02T04:55:58.591Z)

backend\storage\gcs.py:31: error: Module "google.cloud" has no attribute "storage" [attr-
defined]

Found 1 error in 1 file (checked 141 source files)

74. user (2026-07-02T04:56:08.665Z)

```
M backend/config/models.py
M backend/main.py
M backend/utils/paths.py
?? tests/test_startup_output_dir.py
diff --git a/backend/config/models.py b/backend/config/models.py
index 652693a..980e80c 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -272,7 +272,9 @@ class OutputConfig(BaseModel):
     default_highlights_name: str = "highlights.mp4"
     db_name: str = "sports_indexer.db"
     # Directory where the DB, highlight reels, and processed clips are written.
-    # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
+    # An EXPLICIT CLI --output-dir overrides this at startup ? only when the flag
+    # is actually passed (backend/utils/paths.py::resolve_startup_output_dir,
+    # wired in backend/main.py::main).
     output_dir: str = "output"

diff --git a/backend/main.py b/backend/main.py
index 99246fe..6e8377e 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -22,7 +22,7 @@ from dotenv import load_dotenv
# launcher pins it); otherwise env_file_to_load() returns None and load_dotenv(None) keeps the
# exact historical frame-relative walk-up ? byte-identical to the pre-P0a bare load_dotenv().
# stdlib-only import ? keeps this pre-pydantic line light.
-from backend.utils.app_home import env_file_to_load, resolve_output_dir
+from backend.utils.app_home import env_file_to_load

load_dotenv(env_file_to_load())

@@ -78,6 +78,7 @@ from backend.utils.freespace import require_free_bytes # HTTP-507 disk guard
(P
from backend.utils.hashing import compute_video_id # canonical file-identity hashing
from backend.utils.paths import (
    output_base,
+    resolve_startup_output_dir,
    resolve_under_root,
    safe_output_filename,
    validate_input_path,
@@ -1836,8 +1837,9 @@ def main():
    parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
    parser.add_argument(
        "--output-dir",
-        default="output",
-        help="Directory where processed clips/highlights are saved",
+        default=None, # None = not passed -> honor config.json output.output_dir (default
"output")
+        help="Directory where processed clips/highlights are saved. Overrides "
+        "config.json's output.output_dir only when given.",
    )
    parser.add_argument(
        "--setup",
@@ -1924,9 +1926,10 @@ def main():
    out_dir = os.path.dirname(out_path)
    out_name = os.path.basename(out_path)
    else:
-        out_dir = resolve_output_dir(
-            args.output_dir
-        ) # P0a: app-home aware (CWD-identical when unset)
```

```

+         # CLI --output-dir wins when given; otherwise config.json's
+         # output.output_dir (global_config isn't initialized this early).
+         # P0a: app-home aware (CWD-identical when unset).
+         out_dir = resolve_startup_output_dir(args.output_dir, load_app_config())
+         out_path = os.path.join(out_dir, out_filename)
+         out_name = out_filename

@@ -1984,12 +1987,16 @@ def main():
    global_config
    ) # M5: fail fast on an incoherent ACTIVE profile

-    # The CLI --output-dir overrides the configured output directory. It now lives
-    # on the typed model (OutputConfig.output_dir), so every consumer ? including
-    # /api/compile ? reads the same value instead of a write-only runtime hack.
+    # An EXPLICIT CLI --output-dir overrides the configured output directory; without
+    # the flag, config.json's output.output_dir is honored (the argparse default used
+    # to clobber it on every start). It lives on the typed model (OutputConfig.output_dir),
+    # so every consumer ? including /api/compile ? reads the same value instead of a
+    # write-only runtime hack.
+    # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
-    # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
-    global_config.output.output_dir = resolve_output_dir(args.output_dir)
+    # unset and neither knob set, the result is "output" (CWD-relative) ? byte-identical.
+    global_config.output.output_dir = resolve_startup_output_dir(
+        args.output_dir, global_config
+    )
+    os.makedirs(global_config.output.output_dir, exist_ok=True)

    db_path = os.path.join(
diff --git a/backend/utils/paths.py b/backend/utils/paths.py
index 1ed2e7d..979df74 100644
--- a/backend/utils/paths.py
+++ b/backend/utils/paths.py
@@ -3,6 +3,8 @@
 import os
 from typing import Any, Optional

+from backend.utils.app_home import resolve_output_dir
+
def output_base(config: Any) -> str:
    """The configured base dir for a segmenter's LOCAL scratch/staging artifacts
@@ -24,6 +26,26 @@ def output_base(config: Any) -> str:
    return out or "output"

+def resolve_startup_output_dir(cli_output_dir: Optional[str], config: Any) -> str:
+    """The EFFECTIVE output dir at process startup (`backend.main::main`).
+
+    Precedence: an EXPLICIT CLI ``--output-dir`` wins; without it (`None`),
+    config.json's ``output.output_dir`` is honored via :func:`output_base`
+    (default ``"output"`` ? byte-identical to the old always-applied argparse
+    default). Either way P0a app-home rooting applies
+    (:func:`backend.utils.app_home.resolve_output_dir`: a RELATIVE dir roots
+    under ``RALLY_APP_HOME`` when set; an absolute one passes through).
+
+    Regression guard: the argparse default used to be ``"output"`` and was
+    applied UNCONDITIONALLY, silently clobbering the documented
+    ``output.output_dir`` knob (backend/config/models.py::OutputConfig) ? and
+    moving the SQLite DB with it ? on every server/CLI start.
+
+    """
+    if cli_output_dir is not None:
+        return resolve_output_dir(cli_output_dir)
+    return resolve_output_dir(output_base(config))

```



```

+
+
def safe_output_filename(name: str) -> str:
    """Validate an output filename is a BARE name (no dir part, no traversal, not absolute).

```

75. assistant (2026-07-02T04:56:19.261Z)

The diff is minimal and focused. One last sweep ? any other consumer of the `--output-dir` argparse default elsewhere in the repo?

76. user (2026-07-02T04:56:22.363Z)

```

C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-output-dir-config\backend\config\models.py:275:    # An EXPLICIT CLI --output-dir overrides this at startup
? only when the flag
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-output-dir-config\backend\main.py:1839:    "--output-dir",
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-output-dir-config\backend\main.py:1929:    # CLI --output-dir wins when given; otherwise
config.json's
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-output-dir-config\backend\main.py:1990:    # An EXPLICIT CLI --output-dir overrides the configured output
directory; without
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-output-dir-config\backend\main.py:2009:    # or crash) is keyed to db_path, so a different --output-dir/DB
still runs concurrently.
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-output-dir-config\backend\main.py:2029:    f"use a different --output-dir (RALLY_APP_HOME) to
run an independent instance."
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-output-dir-config\backend\pipeline\segmenters\gemini.py:25:    staging follows --output-dir ?
COMPUTE_DECOUPLED_SERVING M3).
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-output-dir-config\backend\utils\single_instance.py:12:which callers co-locate with the DB ? so two
instances on DIFFERENT ``--output-dir`` (different DBs)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-output-dir-config\backend\utils\app_home.py:87:    empty) ? byte-identical to the pre-P0a code that stored
``--output-dir`` as-is (no pathlib
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-output-dir-config\backend\utils\paths.py:12:    chunk-staging / AI-handoff / temp dirs follow ``--output-
dir`` / ``output.output_dir`` instead of
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-output-dir-config\backend\utils\paths.py:32:    Precedence: an EXPLICIT CLI ``--output-dir`` wins; without
it (``None``),

```

77. assistant (2026-07-02T04:56:27.260Z)

Only comments elsewhere ? the flag is defined once. Now verify the branch state and commit with explicit paths.

78. user (2026-07-02T04:56:29.164Z)

fix/output-dir-config-clobber

```
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
warning: in the working copy of 'tests/test_startup_output_dir.py', LF will be replaced by CRLF
the next time Git touches it
M backend/config/models.py
M backend/main.py
M backend/utils/paths.py
A tests/test_startup_output_dir.py
```

79. user (2026-07-02T04:56:44.555Z)

```
[fix/output-dir-config-clobber 8db9175] fix(main): stop the --output-dir argparse default from
clobbering config.json output.output_dir
 4 files changed, 225 insertions(+), 12 deletions(-)
 create mode 100644 tests/test_startup_output_dir.py
8db9175 fix(main): stop the --output-dir argparse default from clobbering config.json
output.output_dir
```

80. assistant (2026-07-02T04:56:49.141Z)

Branch contains only my commit. Push and open the PR.

81. user (2026-07-02T04:56:53.609Z)

```
remote:
remote: Create a pull request for 'fix/output-dir-config-clobber' on GitHub by visiting:
remote:      https://github.com/Khelsutra/badminton-highlight-indexer/pull/new/fix/output-dir-
config-clobber
remote:
branch 'fix/output-dir-config-clobber' set up to track 'origin/fix/output-dir-config-clobber'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
 * [new branch]      fix/output-dir-config-clobber -> fix/output-dir-config-clobber
```

82. user (2026-07-02T04:57:15.749Z)

<https://github.com/Khelsutra/badminton-highlight-indexer/pull/419>

83. assistant (2026-07-02T04:57:22.111Z)

PR #419 is open. Wrapping up with the structured result.

84. user (2026-07-02T04:57:37.876Z)

Structured output provided successfully